TILO

Vyčíslitelnost, složitost, logika a kvantové počítání

Custom skripta podle osnovy 7TILO_1 až 7TILO_14 + XTILO praktikum - auditovaná verze v3

Zdroje: VYSL1, VYSL2, LZUI1, LZUI2 a Quantum Computing via Randomized Algorithms (2025)

Auditovaná verze v3 (XTILO): 28. 9. 2026

Jak tato skripta vznikla

Každý bod osnovy byl nejprve namapován na konkrétní kapitoly zdrojových PDF. Relevantní části byly znovu porovnány se skutečným obsahem zdrojů. Text níže není prosté zkrácené shrnutí: zachovává definice, vztahy, klíčové věty a postupy potřebné pro pochopení a zkoušku. Dlouhé opakující se pasáže a historické komentáře jsou komprimovány. Tam, kde zdroj předpokládá znalost, která je pro pochopení nutná, je přidán výslovně označený blok „Doplnění pro pochopení“.

Jak skripta používat

Skripta jsou uspořádána podle čísel přednášek 7TILO, nikoli podle pořadí původních PDF. Tím se eliminuje nutnost přeskakovat mezi pěti dokumenty. V každé kapitole je uvedena zdrojová mapa a na konci dokumentu je audit pokrytí celé osnovy.

Legenda

Prvek	Význam
Zdrojový výklad	Konsolidovaný a parafrázovaný obsah mapovaných zdrojových částí; tvrzení má být opřeno o uvedený zdroj.
Doplnění pro pochopení	Prerekvizita, vlastní příklad, moderní ekvivalentní formulace nebo spojovací vysvětlení, které není převzato jako tvrzení z hlavního zdroje.
Pozor	Typická chyba, terminologická nuance nebo místo, kde se často zaměňují dva pojmy.
Zdrojová mapa	Konkrétní dokument, kapitola a rozsah, z něhož byla daná část sestavena.

Co znamená „bez ztráty informací“ v tomto dokumentu

Nelze současně výrazně zkrátit pět skript a zachovat doslova každý příklad, každou historickou poznámku a každé cvičení. Proto je zde „bez ztráty“ chápáno ve smyslu pokrytí relevantní látky podle osnovy: žádný bod 7TILO_1 až 7TILO_14 nebyl záměrně vypuštěn. U mapovaných částí jsou zachovány definice, vazby mezi pojmy, klíčové věty a reprezentativní postupy. Příklady a dlouhé důkazy mohou být zkráceny na princip; plný zdroj je vždy dohledatelný podle zdrojové mapy.

Důležitá hranice

Témata, která jsou ve zdrojových PDF, ale nejsou součástí zadané osnovy, nejsou automaticky přepisována. Typicky jde o rozšiřující pasáže, další NP-úplné problémy, historické komentáře, programovou podporu výuky či velké množství cvičení. V dodatku je uvedeno, co bylo záměrně ponecháno mimo hlavní text.

Globální prerekvizity

- Základní práce s množinami, uspořádanými n-ticemi, kartézským součinem, relacemi a funkcemi.
- Čtení jednoduchého pseudokódu a orientace v pojmech stav, přechod, vstup, výstup a konfigurace.
- Základní výroková logika: $\neg$, $\wedge$, $\vee$, $\rightarrow$, pravdivost, splnitelnost a logická platnost.
- Pro predikátovou logiku: proměnná, konstanta, predikát, term, kvantifikátory $\forall$ a $\exists$.
- Pro kvantovou kapitolu: vektory, matice, komplexní čísla, skalární součin a pravděpodobnost. Vše potřebné je stručně zopakováno v kapitole 14.

Obsah podle přednášek

Přednáška	Hlavní téma
7TILO_1	Množiny, relace, funkce, problémy, Turingův stroj a konfigurace
7TILO_2	Kódování TS, ekvivalence, nedeterminismus, univerzální TS, nerozhodnutelnost
7TILO_3	Složitost, asymptotika, P, NP a redukce

7TILO_4	Random Access Machine
7TILO_5	Počáteční funkce, kombinace, kompozice a primitivní rekurze
7TILO_6	Rozsah PRF, tabulkové funkce, μ-rekurzivní a Ackermannova funkce
7TILO_7	Minimalizace, parciálně rekurzivní funkce a vztah k TS
7TILO_8	PL-programy a vyjadřovací síla programovacích jazyků
XTILO praktikum	Semestrální práce: interpret parciálně rekurzivních funkcí, AST, parser, evaluace a testy
7TILO_9	Důkazy, tabla, dedukce a rezoluce ve výrokové logice
7TILO_10	Rezoluce a tabla v predikátové logice
7TILO_11	Opakování
7TILO_12	Axiomatické systémy G, H, G1, H1
7TILO_13	Klauzulární logika a znalostní báze
7TILO_14	Kvantové počítání, Grover, Shor a aplikace

7TILO_1 - Základy, problémy a Turingův stroj

1.1 Množiny, relace a funkce

Množina je kolekce objektů. Základní operace jsou sjednocení $M \cup N$, průnik $M \cap N$, rozdíl a doplněk. Potenční množina $P(M)$ obsahuje všechny podmnožiny M. Kartézský součin $M \times N$ je množina všech uspořádaných dvojic (m,n) s $m \in M$ a $n \in N$.

Relace mezi M a N je libovolná podmnožina $M \times N$. Funkce je speciální relace, v níž je každému prvku definičního oboru přiřazen právě jeden výstup. V teorii vyčíslitelnosti je důležité rozlišovat totální funkci, která je definována pro každý povolený vstup, a parciální funkci, která pro některé vstupy definována být nemusí.

Doplnění pro pochopení

V teorii algoritmů se téměř vše převádí na konečné řetězce symbolů. Proto je užitečné chápat abecedu Σ jako konečnou množinu symbolů, slovo jako konečnou posloupnost symbolů a Σ^* jako množinu všech konečných slov nad Σ včetně prázdného slova ε.

1.2 Co je problém a co znamená vyčíslitelnost

Zdroj formalizuje problém jako vztah mezi instancí a požadovaným výsledkem. Instance i výsledek musí být konečně zapsatelné, tedy kódovatelné řetězcem nad konečnou abecedou. Díky kódování se lze bez ztráty obecnosti soustředit na binární abecedu $\{0,1\}$. Obecný problém tak lze chápat jako totální nebo parciální zobrazení $\Sigma^* \to \Sigma^*$.

Zvláštní a velmi důležitou třídou jsou rozhodovací problémy typu ANO/NE. Pro danou instanci se ptáme, zda má určitou vlastnost. Rozhodnutelný problém má algoritmus, který se pro každý vstup zastaví a odpoví správně. Částečně rozhodnutelný problém má algoritmus, který se zastaví na kladných instancích; na záporné instanci může běžet nekonečně. Nerozhodnutelný problém nemá algoritmus, který by jej správně rozhodl pro všechny instance.

1.3 Intuice Turingova stroje

Turingův stroj (TS) je formální model algoritmu. Má konečný řídicí automat, pásku tvořenou políčky, čtecí/zapisovací hlavu a přechodovou funkci. Na rozdíl od konečného automatu může pásku nejen číst, ale také přepisovat a pohybovat se po ní doleva i doprava. Páska tak funguje jako neomezená pracovní paměť.

Zdroj rozlišuje vstupní abecedu Σ a páskovou abecedu Γ. Pásková abeceda obsahuje vstupní symboly a navíc pracovní symboly včetně blanku #, kterým je vyplněna nepoužitá část pásky.

1.4 Formální definice Turingova stroje

Ve VYSL1 je Turingův stroj definován jako šestice

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- Q - konečná množina stavů.
- Σ - konečná vstupní abeceda; neobsahuje blank #.
- Γ - konečná pásková abeceda, přičemž $\Sigma \subseteq \Gamma$.
- δ - přechodová funkce $\delta : (Q \setminus F) \times \Gamma \to Q \times \Gamma \times \{L,R,N\}$.
- q_0 - počáteční stav.
- F - množina koncových stavů.

Je-li $\delta(q,a)=(q',b,R)$, znamená to: ve stavu q při čtení a přejdi do q', zapiš b a posuň hlavu o jedno políčko doprava. Analogicky L znamená doleva a N bez posunu.

1.5 Konfigurace a výpočet

Konfigurace zachycuje celý okamžitý stav výpočtu: obsah relevantní části pásky, polohu hlavy a vnitřní stav. Zdroj používá zápis uqv: hlava stojí na prvním symbolu části v a stav q je vložen právě před čtený symbol. Například abq_0caab znamená obsah pásky abcaab, stav q_0 a hlavu nad symbolem c.

Relace bezprostředního následování $K_1 \vdash K_2$ vyjadřuje jeden krok výpočtu. Její reflexivní a tranzitivní uzávěr $\vdash^*$ znamená dosažitelnost konečným počtem kroků. Stroj se na vstupu w zastaví, existuje-li konečná posloupnost kroků z počáteční konfigurace q_0w do některé koncové konfigurace.

1.6 Funkce, jazyky a Turingův stroj

Turingův stroj může realizovat parciální funkci: pokud se na vstupu zastaví, obsah pásky reprezentuje výsledek; pokud se nezastaví, hodnota funkce není definována. Pro jazyky rozlišujeme přijímání a rozhodování. Přijímač jazyka L se zastaví na slovech z L, ale mimo L se zastavit nemusí. Rozhodovač se zastaví vždy a rozliší ANO/NE.

Terminologie

Turingovsky rozhodnutelný jazyk = rekurzivní jazyk. Turingovsky částečně rozhodnutelný jazyk = rekurzivně spočetný jazyk. Turingovsky vyčíslitelná parciální funkce je v dalším bloku propojena s parciálně rekurzivními funkcemi.

1.7 Doplnění pro pochopení - mini-příklad průchodu konfiguracemi

Představme si stroj, který přičte k unárnímu zápisu čísla jednu čárku: vstup 111 reprezentuje 3. Stroj jde doprava přes všechny symboly 1. Když poprvé přečte blank #, zapíše 1 a zastaví. Posloupnost konfigurací může vypadat například:

$$q_0111\# \vdash 1q_011\# \vdash 11q_01\# \vdash 111q_0\# \vdash 1111q_f\#$$

1.8 Rozhodovací problém jako jazyk

Každý rozhodovací problém lze identifikovat s jazykem L tvořeným právě těmi kódy instancí, pro které je správná odpověď ANO. Rozhodnout problém tedy znamená rozhodnout členství $w \in L$. Tento pohled propojuje teorii formálních jazyků s vyčíslitelností: algoritmus rozhodovacího problému je současně rozhodovačem určitého jazyka.

1.9 Přesný význam jednoho kroku konfigurace

Je-li konfigurace zapsána $uqav$ a $\delta(q,a)=(q',b,R)$, následující konfigurace má b na místě původního a a stav q' je posunut před následující symbol. Pro pohyb doleva se stav vloží před symbol nalevo od hlavy. Relace $\vdash$ tak není „nová operace stroje“, ale formální zápis výsledku aplikace δ na jednu konfiguraci.

Rekurzivní vs. rekurzivně spočetný jazyk

Je-li L rekurzivní, existuje TS, který se zastaví na každém slově a správně rozhodne členství. Je-li L pouze rekurzivně spočetný, existuje přijímač: slova z L nakonec přijme, ale na slově mimo L může běžet navždy. Každý rekurzivní jazyk je rekurzivně spočetný, opačně to neplatí.

Co musíš umět

Popsat komponenty TS, přečíst přechod $\delta(q,a)$, vysvětlit konfiguraci, zapsat několik kroků výpočtu, odlišit přijímání jazyka od jeho rozhodování a interpretovat rozhodovací problém jako jazyk.

Zdrojová mapa

VYSL1: kap. 1.1 Množiny, 1.4 Relace, 1.5 Funkce; kap. 2 Vyčíslitelnost a problém; kap. 3.1-3.2 Turingovy stroje (v obsahu přibližně s. 10-45).

7TILO_2 - Kódování, varianty Turingova stroje a nerozhodnutelnost

2.1 Proč kódovat Turingův stroj

Kódování převádí popis stroje na konečné slovo, typicky nad {0,1}. Tím se Turingův stroj sám stává daty, se kterými může pracovat jiný Turingův stroj. Tento krok je klíčový pro univerzální stroj i pro důkazy nerozhodnutelnosti, protože umožňuje formulovat vstupy typu „kód stroje M spolu se slovem w“.

2.2 Ekvivalentní a modifikované Turingovy stroje

Zdroj probírá několik modifikací klasického modelu. Může jít například o více pásek, odlišné konvence pro pohyb hlavy nebo jiné technické úpravy. Podstatný závěr není, že jsou stroje konstrukčně stejné, ale že mají stejnou výpočetní sílu: co spočítá jedna rozumná varianta, lze simulovat druhou.

Doplnění pro pochopení

Ekvivalence výpočetních modelů se obvykle prokazuje simulací. Neřešíme, zda simulace zachová počet kroků 1:1; pro vyčíslitelnost stačí, že každý konečný výpočet lze převést na konečný výpočet druhého modelu. U teorie složitosti už režie simulace důležitá být může.

2.3 Nedeterministický Turingův stroj

U deterministického TS je pro dvojici (stav, čtený symbol) určen nejvýše jeden následující krok. Nedeterministický Turingův stroj (NTS) může mít více možných pokračování. Výpočet proto netvoří jedinou posloupnost konfigurací, ale strom výpočtů. Pro přijímání se používá existenční interpretace: vstup je přijat, existuje-li alespoň jedna přijímající výpočetní větev.

Pro samotnou vyčíslitelnost nedeterminismus nepřidává novou výpočetní sílu: deterministický stroj může systematicky simulovat větve nedeterministického výpočtu. V teorii složitosti je však rozdíl zásadní a vede ke třídě NP.

2.4 Univerzální Turingův stroj

Univerzální Turingův stroj U dostane na vstupu kód jiného stroje M a vstup w a simuluje výpočet M na w. Schematicky:

$$U(\langle M\rangle, w) \text{ simuluje } M(w)$$

Je to formální analogie univerzálního programovatelného počítače: program není zabudován do hardwaru, ale je součástí vstupu.

2.5 Nerozhodnutelné problémy

Nerozhodnutelnost neznamená, že neznáme dostatečně chytrý algoritmus. Znamená, že žádný algoritmus daného modelu - a podle Church-Turingovy téze žádný algoritmus v intuitivním smyslu - nemůže problém rozhodnout pro všechny vstupy.

Problém zastavení

Klasickým příkladem je Halting Problem. Instance obsahuje kód stroje M a slovo w. Otázka zní, zda se M na w zastaví. Tento problém není rozhodnutelný.

$$HP = \{ \langle M,w\rangle \mid M \text{ se na } w \text{ zastaví} \}$$

Důkaz využívá sebe-referenci a spor: předpokládáme existenci univerzálního rozhodovače zastavení a s jeho pomocí zkonstruujeme stroj, který se na vlastním kódu chová opačně, než rozhodovač předpoví.

Redukce problémů

Chceme-li ukázat nerozhodnutelnost nového problému B, často vezmeme známý nerozhodnutelný problém A a efektivně převedeme jeho instance na instance B tak, aby odpověď zůstala zachována. Kdyby existoval rozhodovač B, získali bychom přes převod rozhodovač A - spor.

$$A \leq B \text{ a } A \text{ je nerozhodnutelný } \Rightarrow B \text{ je nerozhodnutelný}$$

Směr redukce

Při důkazu, že B je těžký, převádíme známý těžký problém A NA B. Obrácený směr by požadovaný závěr nedal.

Postův korespondenční problém a další nerozhodnutelné problémy

VYSL1 používá jako další reprezentativní problém Postův korespondenční problém: z dvojic slov hledáme posloupnost indexů, která po konkatenaci vytvoří nahoře i dole stejné slovo. Slouží jako nástroj k přenosu nerozhodnutelnosti do problémů formálních jazyků.

2.6 Jak vypadá kódování přechodů

Konkrétní binární kód není z hlediska teorie jedinečný. Stačí, aby existovalo efektivní jednoznačné kódování stavů, páskových symbolů a jednotlivých přechodů a aby šel celý seznam přechodů rozdělit a znovu dekódovat. Stroj pak reprezentujeme slovem ⟨M⟩. Důležitá vlastnost je efektivnost obou směrů: z M vytvořit ⟨M⟩ a z platného kódu rekonstruovat popis M.

2.7 Důkaz nerozhodnutelnosti HP - kostra argumentu

1. Předpokládej, že existuje rozhodovač H(⟨M⟩,w), který vždy správně určí, zda se M na w zastaví.
2. Zkonstruuj stroj D, který na vstupu ⟨M⟩ zavolá H(⟨M⟩,⟨M⟩). Pokud H předpoví zastavení, D záměrně běží navždy; pokud H předpoví nezastavení, D se zastaví.
3. Pusť D na jeho vlastním kódu ⟨D⟩. Pokud H řekne „zastaví“, definice D způsobí nezastavení. Pokud řekne „nezastaví“, D se zastaví.
4. Obě možnosti vedou ke sporu, takže H nemůže existovat.

2.8 Ukázka redukce HP → UHP podle VYSL1

Uniform Halting Problem se ptá, zda se zadaný stroj zastaví na každém vstupu. VYSL1 převádí instanci (M,w) problému HP na stroj M′ takto: M′ nejprve otestuje svůj aktuální vstup. Je-li vstup roven pevnému slovu w, spustí simulaci stroje M; je-li vstup od w odlišný, ihned se zastaví. Proto se M′ zastaví na všech vstupech právě tehdy, když se původní M zastaví na w. Kdyby tedy byl UHP rozhodnutelný, byl by rozhodnutelný i HP - spor.

Co musíš umět

Vysvětlit smysl kódování TS, rozdíl deterministický/nedeterministický TS, princip univerzálního TS, formulovat problém zastavení, reprodukovat kostru diagonalizačního důkazu a správný směr redukčního důkazu.

Zdrojová mapa

VYSL1: 3.3 Kódování TS, 3.4 Ekvivalenty a modifikace, 3.5 Nedeterministické TS, 3.6 Univerzální TS, kap. 4 Nerozhodnutelné problémy (s. 46-102 v obsahu).

7TILO_3 - Složitost, asymptotické odhady, P a NP

3.1 Od vyčíslitelnosti ke složitosti

Vyčíslitelnost řeší otázku „lze problém algoritmicky vyřešit?“. Složitost se ptá „kolik zdrojů je k řešení potřeba?“. Dva algoritmy mohou řešit tentýž problém, ale mít dramaticky odlišnou časovou nebo paměťovou náročnost.

3.2 Časová složitost Turingova stroje

Časová složitost konkrétního výpočtu je počet elementárních kroků do zastavení. Časová složitost stroje M se bere jako funkce délky vstupu n a používá nejhorší případ:

$$T_M(n) = \max \{ \text{počet kroků } M \text{ na } w \mid |w| = n \}$$

3.3 Asymptotické odhady

Přesný počet kroků bývá méně důležitý než rychlost růstu. Zdroj zavádí O, o, Θ a dolní odhad. Prakticky nejdůležitější jsou:

- $f(n) \in O(g(n))$: pro dost velká n je f shora omezena konstantním násobkem g.
- $f(n) \in \Theta(g(n))$: f a g mají stejný asymptotický řád - každá je O druhé.
- $f(n) \in o(g(n))$: f roste asymptoticky přísně pomaleji než g.
- Dolní odhad vyjadřuje, že výpočet nemůže být asymptoticky rychlejší než daná funkce v příslušném smyslu.

Doplnění pro pochopení

V moderních učebnicích se běžně používá $\Omega(g)$ jako asymptotický dolní odhad „od určitého n dál“. VYSL1 používá notaci Ω^{∞} pro dolní odhad splněný pro nekonečně mnoho n. Při zkoušce respektuj značení z přednášky.

Typické růsty od menších k větším: $\log n$, n, $n \log n$, n^2, n^3, 2^n. Konstantní násobky a nižší členy v O-notaci ignorujeme.

3.4 Složitost problému a třídy složitosti

Problém neztotožňujeme s jedním algoritmem. Jeho složitost souvisí s nejlepším algoritmem, který jej řeší. Zdroj zavádí třídy T(f): problémy řešitelné v čase O(f). Strukturální teorie složitosti pak sleduje vztahy mezi celými třídami.

3.5 PTIME / P

PTIME (zkráceně P) je třída rozhodovacích problémů řešitelných deterministickým Turingovým strojem v polynomiálním čase O(n^k) pro nějakou konstantu k. Polynomiální čas se používá jako robustní teoretická aproximace praktické zvládnutelnosti.

$$P = \bigcup_k T(n^k)$$

3.6 Polynomiální redukce

Polynomiální redukce $A \leq_p B$ je algoritmický převod instance A na instanci B v polynomiálním čase se zachováním odpovědi ANO/NE. Pokud $A \leq_p B$ a $B \in P$, potom $A \in P$. Redukce jsou základním nástrojem porovnávání obtížnosti problémů.

3.7 NPTIME / NP - formulace ve VYSL1

VYSL1 definuje rozhodování nedeterministickým Turingovým strojem tak, že všechny jeho výpočty jsou konečné a vydávají ANO nebo NE; pro instanci I existuje alespoň jeden výpočet vydávající ANO právě tehdy, když je správná odpověď ANO. Složitost NTS pro slovo w je ve zdroji definována jako délka nejkratšího výpočtu vydávajícího ANO, pokud takový existuje; jinak jako délka nejkratšího výpočtu vydávajícího NE. Třída problémů rozhodnutelných tímto způsobem v polynomiálním čase je označena NPTIME neboli NP.

$$\text{PTIME} \subseteq \text{NPTIME} \subseteq \text{PSPACE}$$

Otevřená otázka P = NP se ptá, zda je PTIME rovno NPTIME.

Doplnění pro pochopení - moderní ekvivalentní pohled

V běžných současných učebnicích se NP často charakterizuje pomocí polynomiálně dlouhého certifikátu, který lze deterministicky ověřit v polynomiálním čase. Tento pohled je standardně ekvivalentní obvyklé nedeterministické definici, ale není to formulace, kterou používá VYSL1.

3.8 NP-těžkost a NP-úplnost

Problém je NP-těžký, pokud se na něj polynomiálně redukuje každý problém z NP. Je NP-úplný, pokud je zároveň v NP. Zdroj uvádí Cookovu větu: SAT je NP-úplný. Z jednoho NP-úplného problému se pak NP-úplnost dalších dokazuje řetězením redukcí.

$$\text{NP-úplný} = \text{NP-těžký} + \text{členství v NP}$$

3.9 Prostorová složitost a další třídy

Vedle času lze měřit použitou paměť. Zdroj připomíná třídy PSPACE a EXPSPACE a také EXPTIME. Důležitý orientační řetězec je PTIME $\subseteq$ PSPACE $\subseteq$ EXPTIME $\subseteq$ EXPSPACE. Hlavní osnova se soustředí na P a NP, ale tento řetězec pomáhá chápat, že čas a prostor jsou různé zdroje.

3.10 Doplnění pro pochopení - jak číst O-notaci na příkladu

Je-li $T(n)=3n^2+10n+20$, pak $T \in O(n^2)$ a současně $T \in \Theta(n^2)$. Tvrzení $T \in O(n^3)$ je také pravdivé, ale méně přesné. O-notace proto sama neříká „přesnou složitost“, nýbrž horní asymptotickou hranici.

3.11 Doplnění pro pochopení - certifikáty a NP

U rozhodovacího problému v NP lze pro ANO-instanci předložit certifikát polynomiální délky a deterministicky ho ověřit v polynomiálním čase. U SAT je certifikátem ohodnocení proměnných; ověření spočívá v dosazení a vyhodnocení formule. Tento pohled je prakticky ekvivalentní nedeterministickému „uhodnutí“ správného certifikátu.

3.12 Doplnění pro pochopení - praktický postup důkazu NP-úplnosti

1. Ukaž, že nový problém B patří do NP - popiš certifikát a polynomiální ověření.
2. Vyber známý NP-těžký/NP-úplný problém A.
3. Sestroj polynomiální redukci $A \leq_p B$.
4. Dokaž obě implikace: instance A je ANO právě tehdy, když vytvořená instance B je ANO.

Co musíš umět

Definovat časovou složitost v nejhorším případě, vysvětlit O a Θ, definovat P a NP ve verzi NTM i certifikátu, vysvětlit polynomiální redukci a rozlišit NP-těžký vs. NP-úplný problém.

Zdrojová mapa

VYSL1: kap. 8 Složitost, zejména 8.1-8.6 (s. 140-150); rozšiřující příklady NP-úplných problémů v 8.7.

7TILO_4 - Random Access Machine (RAM)

4.1 Proč další výpočetní model

Turingův stroj je minimalistický a teoreticky elegantní, ale pro programátora je vzdálen reálnému procesoru. RAM (Random Access Machine) model pracuje s celými čísly, registry a instrukcemi a dovoluje náhodný přístup do pracovní paměti. Přesto má podle zdroje stejnou výpočetní sílu jako Turingův stroj.

4.2 Součásti RAM stroje

- Programová jednotka - konečná posloupnost očíslovaných instrukcí.
- Neomezená pracovní paměť - adresované buňky/registry, každý může obsahovat celé číslo.
- Vstupní páska - sekvenční čtení celých čísel.
- Výstupní páska - sekvenční zápis celých čísel.
- Centrální jednotka - programový čítač (IC) a aritmeticko-logická jednotka (ALU).

4.3 Adresování a operandy

Zdroj rozlišuje přímou konstantu, obsah konkrétního registru a nepřímé/indexované adresování. Hlavní myšlenka náhodného přístupu je, že přístup k pracovní buňce není realizován sekvenčním posouváním hlavy jako u TS.

4.4 Program a běh

Výpočet začíná první instrukcí; programový registr určuje následující instrukci. Běžné instrukce načítají vstup, zapisují výstup, přesouvají data, provádějí aritmetiku a podmíněné/nepodmíněné skoky. Výpočet končí instrukcí HALT.

4.5 RAM vs. Turingův stroj

Vlastnost	Turingův stroj	RAM
Data	symboly konečné abecedy	celá čísla
Paměť	páska, přístup sekvenční	adresované registry, náhodný přístup
Řízení	stav + přechodová funkce	čítač instrukcí + program
Aritmetika	musí být realizována z elementárních přechodů	předpokládá se ALU/instrukce
Výpočetní síla	univerzální	ekvivalentní TS

Pozor na model ceny

Při analýze složitosti RAM je důležité, co považujeme za jednotkovou operaci. Pokud by operace s libovolně dlouhými čísly stála vždy 1 krok, model by mohl uměle podhodnocovat skutečnou náročnost. V teorii se proto rozlišují různé cenové modely.

4.6 Doplnění pro pochopení - mini-příklad

Program pro součet vstupních čísel může číst hodnoty ze vstupní pásky, akumulovat součet v registru a po dosažení ukončovací podmínky zapsat součet na výstup. V TS by stejná úloha vyžadovala explicitní kódování čísel do symbolů a konstrukci algoritmu pro sčítání.

4.7 Instrukční sada podle VYSL1

Skupina	Instrukce	Význam
Vstup/výstup	READ, WRITE	READ načte další vstup do pracovního registru; WRITE zapíše

		pracovní registr na výstup.
Paměť	LOAD, STORE	LOAD načte operand do pracovního registru; STORE zapíše pracovní registr do paměti.
Aritmetika	ADD, odečtení, MUL, DIV	VYSL1 u odečtení v tabulce tiskne znovu „STORE“, ale popis instrukce říká, že se od pracovního registru odečte operand; význam tedy odpovídá instrukci SUB.
Skoky	JUMP, JZERO, JGTZ	Nepodmíněný skok; skok při nule; skok při kladné hodnotě.
Zastavení	HALT	Regulérní ukončení programu.

4.8 Tři druhy operandů

Zdroj používá zápis =i pro bezprostřední konstantu i, i pro obsah paměťové buňky s adresou i a *i pro nepřímý/indexovaný přístup. Tím se RAM přibližuje skutečnému strojovému kódu: program může pracovat s konstantou, pevnou adresou i adresou vypočítanou z registru.

4.9 Příklad - výměna hodnot registrů

Chceme-li vyměnit hodnoty R1 a R2, potřebujeme pomocný registr R3: LOAD 1; STORE 3; LOAD 2; STORE 1; LOAD 3; STORE 2. Na tomto příkladu je vidět, že pracovní registr funguje podobně jako akumulátor a STORE je klíčová instrukce pro změnu paměti.

Co musíš umět

Popsat architekturu RAM, vysvětlit náhodný přístup, znát hlavní skupiny instrukcí, rozlišit přímé a nepřímé adresování, porovnat RAM s TS a přečíst jednoduchý RAM program.

Zdrojová mapa

VYSL1: kap. 6 Model RAM (Random Access Machine), 6.1 Prvky RAM stroje a 6.2 Aplikace RAM (s. 112-131).

7TILO_5 - Počáteční funkce a primitivní rekurze

5.1 Od programů k funkcím

Teorie rekurzivních funkcí popisuje vyčíslitelnost bez odkazu na konkrétní stroj. Po vhodném kódování dat lze pracovat s parciálními funkcemi tvaru f : N^m → N^n. Parciální funkce nemusí být definována pro všechny vstupy; totální funkce definována je.

5.2 Počáteční funkce

VYSL2 staví hierarchii na velmi jednoduchých funkcích:

- Nulová funkce ζ: ζ() = 0.
- Funkce následníka σ: σ(x) = x + 1.
- Projekce π^n_k: horní index n udává rozměr vstupní n-tice a dolní index k vybíranou složku; např. $\pi^3_2(7,6,4)=6$.

Samy o sobě jsou jednoduché, ale po uzavření vůči několika konstrukčním operacím z nich vznikne velmi bohatá třída funkcí.

5.3 Kombinace

Pro f : N^k → N^m a g : N^k → N^n je kombinace f × g funkcí, která na stejném vstupu vypočítá oba výstupy a spojí je do (m+n)-tice:

$$(f \times g)(x) = (f(x), g(x))$$

5.4 Kompozice

Je-li výstup f vhodným vstupem pro g, kompozice znamená „nejprve f, potom g“:

$$(g \circ f)(x) = g(f(x))$$

> **Pozor**
>
> Pořadí symbolů v g ∘ f je opačné vůči pořadí provádění: jako první se vyhodnotí f.

5.5 Primitivní rekurze

Primitivní rekurze vytváří funkci pomocí základního případu a rekurentního kroku. Typický tvar je:

$$f(x,0)=g(x)$$

$$f(x,y+1)=h(x,y,f(x,y))$$

Pro výpočet f(x,y) se postupně spočítá f(x,0), f(x,1), ..., f(x,y). Proto se konstrukce vždy po konečně mnoha krocích zastaví, jsou-li g a h totální.

5.6 Primitivně rekurzivní funkce (PRF)

Třída primitivně rekurzivních funkcí je nejmenší třída obsahující počáteční funkce a uzavřená na kombinaci, kompozici a primitivní rekurzi. Zdroj zdůrazňuje zásadní vlastnost: každá PRF je totální.

$$\text{počáteční funkce} + \text{kombinace} + \text{kompozice} + \text{primitivní rekurze} \Rightarrow \text{PRF}$$

Známé aritmetické funkce jako sčítání, násobení, předchůdce, omezené odčítání (monus), charakteristické funkce jednoduchých relací a řada dalších funkcí lze vyjádřit primitivní rekurzí.

5.7 Příklad: sčítání

Sčítání lze chápat jako opakované použití následníka:

$$plus(x,0)=x$$

$$plus(x,y+1)=\sigma(plus(x,y))$$

Tento zápis je přesně ve tvaru primitivní rekurze. Z něj je vidět nejen výpočet, ale i důvod, proč je plus primitivně rekurzivní.

5.8 Další standardní PRF konstrukce

Ze základních konstrukcí lze odvodit řadu pomocných funkcí. Konstantní funkci K^n_c lze sestavit opakovaným použitím následníka a projekcí. Předchůdce pred lze definovat primitivní rekurzí tak, aby pred(0)=0 a pred(y+1)=y. Omezené odčítání monus(x,y)=max(x-y,0) vznikne opakovaným použitím předchůdce.

$$monus(x,0)=x, \quad monus(x,y+1)=pred(monus(x,y))$$

5.9 Doplnění pro pochopení - primitivní rekurze není totéž co programová rekurze

Pojem „primitivní rekurze“ označuje přesné matematické schéma s parametrem, který se krok po kroku snižuje k základnímu případu. Běžná rekurzivní funkce v programovacím jazyce může být mnohem obecnější a může se nezastavit. Proto z toho, že program používá rekurzi, automaticky neplyne, že počítá PRF.

Co musíš umět

Vyjmenovat počáteční funkce, vysvětlit kombinaci a kompozici, přečíst schéma primitivní rekurze, sestavit jednoduché plus/pred/monus a zdůvodnit, proč je každá primitivně rekurzivní funkce totální.

Zdrojová mapa

VYSL2: kap. 1 Parciální funkce, 1.1 Počáteční funkce, 1.2 Primitivně rekurzivní funkce (s. 11-20).

7TILO_6 - Rozsah PRF, tabulkové, μ-rekurzivní a Ackermannova funkce

6.1 Jak bohatá je třída PRF

VYSL2 ukazuje, že primitivně rekurzivní jsou prakticky všechny běžné elementární celočíselné funkce používané v příkladech teorie algoritmů: konstantní funkce, předchůdce, monus, sčítání, násobení, test rovnosti, negace charakteristické funkce a další.

6.2 Tabulkové funkce

Tabulková funkce je zde funkce, pro kterou explicitně uvedeme konečný počet zvláštních vstupů a jejich výstupů a všem ostatním vstupům přiřadíme společnou výchozí hodnotu. Zdroj ukazuje, že takové funkce lze sestavit z charakteristických funkcí, konstant, součtů a dalších PRF, a jsou tedy také primitivně rekurzivní.

6.3 Mezera nad PRF

Z toho, že PRF obsahuje obrovské množství funkcí, neplyne, že obsahuje všechny vyčíslitelné totální funkce. Právě tato hranice je hlavním tématem druhé části kapitoly.

6.4 Ackermannova funkce

Klasickým příkladem totální vyčíslitelné funkce, která není primitivně rekurzivní, je Ackermannova funkce. Zdroj používá definici:

$$A(0,y)=y+1$$

$$A(x+1,0)=A(x,1)$$

$$A(x+1,y+1)=A(x, A(x+1,y))$$

Důležitý není ruční výpočet velkých hodnot, ale její role v hierarchii: roste tak rychle a má tak hlubokou rekurzivní strukturu, že uniká třídě PRF, přesto je algoritmicky vyčíslitelná a totální.

6.5 μ-rekurzivní funkce ve značení zdroje

VYSL2 označuje třídu vyčíslitelných totálních funkcí jako μ-rekurzivní funkce. Je třeba nezaměnit toto označení s operátorem minimalizace μ, který je zaveden až v následující kapitole jako způsob konstrukce parciálně rekurzivních funkcí.

> **Terminologická nuance**
>
> V literatuře se pojmy „μ-recursive“, „general recursive“ a „partial recursive“ používají různě podle tradice. Pro zkoušku je bezpečné držet se terminologie daných skript: PRF ⊊ vyčíslitelné totální funkce (zde μ-rekurzivní) ⊆ parciálně rekurzivní funkce.

6.6 Diagonalizační myšlenka

Zdroj ukazuje existenci totální vyčíslitelné funkce mimo PRF také diagonalizačním argumentem. PRF lze seřadit $f_1,f_2,...$ podle konečných zápisů jejich definic. Funkce $f(n)=f_n(n)+1$ se v bodě n liší od n-té PRF, takže nemůže být žádnou z nich. Tento typ argumentu se později opakuje v různých podobách teoretické informatiky.

6.7 Charakteristické funkce a logické operace

Relace nad N lze reprezentovat charakteristickou funkcí s hodnotami 0/1. Díky PRF konstrukcím lze vytvořit test rovnosti eq(x,y), jeho negaci a kombinovat podmínky aritmetickými funkcemi. To je důležité pro tabulkové funkce: každou z konečně mnoha zvláštních hodnot lze „zapnout“ charakteristickou funkcí příslušného vstupu.

6.8 Doplnění pro pochopení - proč Ackermannova funkce překračuje PRF

Intuice je, že každá konkrétní PRF má předem konečně zanořenou konstrukci primitivních rekurzí. Ackermannova funkce používá úroveň rekurze, která sama roste se vstupem, a překoná rychlost růstu každé pevně zvolené PRF. Formální důkaz je hlubší; ke zkoušce je klíčový význam této funkce jako separačního příkladu.

Co musíš umět

Vysvětlit, proč jsou tabulkové funkce PRF, znát význam charakteristických funkcí, Ackermannovy funkce a diagonalizačního argumentu a chápat, že třída PRF neobsahuje všechny totální vyčíslitelné funkce.

Zdrojová mapa

VYSL2: kap. 2 Rozsah primitivně rekurzivních funkcí, zejména 2.1 Příklady PRF a 2.2 Nad třídou PRF (s. 21-32).

7TILO_7 - Minimalizace, parciálně rekurzivní funkce a jejich vztah k TS

7.1 Operátor minimalizace

Minimalizace dovoluje hledat nejmenší y, pro které určitá funkce nabude hodnoty 0. Ve skriptech je definována:

$$f(x) = \mu y [g(x,y) = 0]$$

Výsledek je nejmenší y, pro které g(x,y)=0, přičemž g(x,z) musí být definována pro všechna z<y. Pokud takové y nenajdeme nebo dřívější výpočet g není definován, může být f nedefinována. Právě zde se přirozeně objevují skutečně parciální funkce.

7.2 Parciálně rekurzivní funkce

Třída parciálně rekurzivních funkcí vzniká z počátečních funkcí uzavřením na kombinaci, kompozici, primitivní rekurzi a minimalizaci. Oproti PRF už není garantována totalita.

$$\text{PRF} + \text{minimalizace } \mu \Rightarrow \text{parciálně rekurzivní funkce}$$

7.3 Kódování čísel na pásce TS

Aby bylo možné porovnat funkční formalismus s Turingovými stroji, skripta reprezentují nezáporná čísla binárně na pásce a n-tice jako posloupnost binárních čísel oddělených blanky. Tím může TS počítat funkce N^m → N^n.

7.4 Každá parciálně rekurzivní funkce je turingovsky vyčíslitelná

Důkaz se skládá po konstrukčních operacích. Nejprve se ukáže vyčíslitelnost počátečních funkcí. Potom:

- kombinace se realizuje paralelním/sériovým výpočtem dvou strojů a spojením výsledků;
- kompozice se realizuje spuštěním stroje pro vnitřní funkci a následně stroje pro vnější funkci;
- primitivní rekurze se simuluje postupným výpočtem hodnot pro 0,1,...,y;
- minimalizace se simuluje zkoušením y=0,1,2,... až do nalezení první nuly - a nemusí skončit.

7.5 Opačný směr: TS jako parciálně rekurzivní proces

Skripta nekončí tvrzením „funkce lze naprogramovat TS“, ale kódují samotný výpočet TS aritmeticky. Obsah pásky se interpretuje jako celé číslo v základu b=|Γ|. Stavy jsou očíslovány celými čísly a poloha hlavy je také číslo.

Konfigurace stroje je pak reprezentována trojicí:

$$(w, p, n)$$

- w - číslo reprezentující obsah pásky;
- p - číslo reprezentující aktuální stav;
- n - číslo pozice hlavy.

Následující konfigurace se popisuje aritmetickými funkcemi typu nextstate, nexttape a nexthead. Jejich kombinací vznikne funkce step : $N^3 \rightarrow N^3$ simulující jeden krok TS. Tím lze celý výpočet převést do jazyka rekurzivních funkcí.

7.6 Church-Turingova vazba

Výsledkem obou směrů je silná shoda mezi různými formalismy vyčíslitelnosti: to, co umíme vyjádřit parciálně rekurzivní funkcí, umíme vyčíslit Turingovým strojem, a chování Turingova stroje lze aritmeticky reprezentovat parciálně rekurzivními funkcemi. Tato ekvivalence podporuje Church-Turingovu tezi.

7.7 Doplnění pro pochopení - příklad minimalizace

Nechť g(x,y)=|x-y|. Potom μy[g(x,y)=0]=x, protože první y s nulovou hodnotou je právě x. Naproti tomu pokud g nikdy nuly nenabude, minimalizace není definována. Tento jednoduchý kontrast vysvětluje, proč μ přechází od garantovaně totálních PRF k parciálním funkcím.

7.8 Simulace jednoho kroku TS aritmeticky

Máme-li kód konfigurace (w,p,n), funkce cursym získá z čísla w symbol pod hlavou. Tabulkové funkce mohou z dvojice (p,cursym) získat nový stav a nový symbol. Aritmetická operace upraví příslušnou „číslici“ kódu pásky a nexthead upraví n. Všechny tyto části jsou ve VYSL2 sestaveny z parciálně rekurzivních funkcí pomocí kompozice a kombinace, takže i step je parciálně rekurzivní. Iterací step se aritmeticky popisuje celý běh stroje.

7.9 Dva směry ekvivalence

Směr	Co se dokazuje	Hlavní technika
parciálně rekurzivní funkce → TS	každou konstrukci parciálně rekurzivní funkce umí simulovat stroj	skládání strojů, iterace, minimalizace
TS → parciálně rekurzivní funkce	výpočet TS lze aritmeticky reprezentovat parciálně rekurzivními funkcemi	číselný kód pásky/stavu/hlavy + funkce step/run/minimalizace času zastavení

Co musíš umět

Použít definici minimalizace na příkladu, vysvětlit vznik parciálních funkcí, popsat reprezentaci n-tic na pásce, oba směry ekvivalence parciálně rekurzivní funkce ↔ Turingovy stroje a význam číselného kódování konfigurace (w,p,n).

Zdrojová mapa

VYSL2: kap. 3 Parciálně rekurzivní funkce, 3.1-3.4 (s. 33-50). Zvlášť 3.4 pro číselnou reprezentaci konfigurace a funkci step.

7TILO_8 - PL programy a vyjadřovací síla programovacích jazyků

8.1 Proč další formalismus algoritmu

PL-programy a skeletový programovací jazyk představují výpočet z pohledu programátora. Cílem není vytvořit pohodlný praktický jazyk, ale co nejmenší jazyk, na kterém lze studovat hranici vyčíslitelnosti.

8.2 PL-programy ve VYSL1

VYSL1 definuje PL-program jako sekvenci příkazů nad identifikátory/proměnnými. Jádro jazyka používá velmi jednoduché konstrukce: nastavení na nulu, inkrement, cyklus a v rozšířené variantě návěští a GOTO. Důležitá je výpočetní síla těchto variant.

- Bez neomezeného skoku má jazyk charakter omezených iterací a odpovídá primitivně rekurzivním výpočtům.
- Přidání mechanismu umožňujícího neomezené opakování/skok dovolí vyjádřit obecné parciální výpočty.

8.3 Skeletový jazyk ve VYSL2

VYSL2 používá obdobný minimalistický programovací jazyk pro demonstraci Church-Turingovy ekvivalence. Disponuje proměnnými nad N a příkazy pro nulování, inkrementaci/dekrementaci a while-cyklus. Potom konstrukčně ukazuje, že v něm lze implementovat počáteční funkce i operace kombinace, kompozice, primitivní rekurze a minimalizace.

8.4 Kompozice jako skládání programů

Je-li program F realizací funkce f a program G realizací g, kompozici $g \circ f$ lze získat spuštěním F a následně G s vhodným propojením výstupních a vstupních identifikátorů. Tato jednoduchá myšlenka je programovým protějškem funkční kompozice.

8.5 Minimalizace jako while

Operátor μ lze implementovat tak, že nastavíme y=0, opakovaně počítáme g(x,y) a dokud není výsledek 0, y zvýšíme. Jestliže se nula nikdy nenajde nebo některý vnitřní výpočet nedoběhne, program se nezastaví. To přesně odpovídá parciálnosti.

8.6 Vyjadřovací síla

Závěr VYSL2: jednoduchý jazyk s přirozenými čísly, změnou jejich hodnot a while-cyklem má dostatečnou vyjadřovací sílu pro každý algoritmicky řešitelný problém v Church-Turingově smyslu. Další jazykové konstrukce mohou zásadně zlepšit čitelnost a efektivitu programování, ale ne nutně teoretickou vyčíslitelnost.

Doplnění pro pochopení

„Silnější jazyk“ může znamenat dvě různé věci: (1) vyšší teoretická vyjadřovací síla - umí spočítat více funkcí; (2) praktičtější expresivita - stejnou funkci zapíšeme mnohem kratším a srozumitelnějším programem. Turingovsky úplné běžné jazyky se liší hlavně ve druhém smyslu.

8.7 Přesný seznam PL prvků ve VYSL1

Prvek	Význam
X := 0	vynulování proměnné
INC X	inkrementace X := X+1
LOOP X ... ENDLOOP	provedení těla právě X-krát
L	návěští/bod programu
GOTO L	skok na návěští

INPUT X1,X2,... / OUTPUT Y	specifikace vstupů a výstupu

Zdroj explicitně uvádí, že bez GOTO mají PL-programy stejnou výpočetní sílu jako PRF; s GOTO odpovídají obecnějším rekurzivním funkcím. Libovolný PL-program lze rovněž simulovat Turingovým strojem.

8.8 Příklad PL programu pro součet

```
INPUT X1,X2;  Y:=0;  LOOP X1 INC Y ENDLOOP;  LOOP X2 INC Y ENDLOOP;  OUTPUT Y
```

Program názorně ukazuje ekvivalenci iterace a primitivní rekurze: počet iterací je znám z hodnoty proměnné na začátku cyklu. Neomezený GOTO/while naopak dovoluje běh, jehož délku nelze předem ohraničit argumentem primitivní rekurze.

Co musíš umět

Popsat přesné základní příkazy PL, vysvětlit vztah LOOP ↔ PRF a GOTO/while ↔ obecné parciální výpočty, přečíst jednoduchý PL program a formulovat závěr o vyjadřovací síle.

Zdrojová mapa

VYSL1: 7.2 PL-programy (s. 136-139). VYSL2: kap. 4 Vyjadřovací síla programovacích jazyků, 4.1-4.3 (s. 51-62).

XTILO praktikum - Semestrální práce: interpret parciálně rekurzivních funkcí

Rozsah této kapitoly

Tato praktická kapitola propojuje teorii 7TILO_5 až 7TILO_8 s oficiálním zadáním semestrální práce pro studenty XTILO. Požadavky projektu jsou převyprávěny z oficiální stránky zadání; návrhové vzory, pseudokód a odvození AST jsou označeny jako doporučená implementační konstrukce.

Oficiální zadání: web kurzu 7TILO / XTILO, stránka „Semestrální práce - Interpret PRF“ (stav ověřen 28. 9. 2026). Zadání vyžaduje univerzální interpret, v němž funkce nejsou speciální metody typu add() nebo multiply(), ale formální syntaktické stromy složené ze základních konstrukcí teorie parciálně rekurzivních funkcí.

P.1 Co je cílem projektu

Jádrem projektu je oddělit syntaxi funkce od její evaluace. AST pouze popisuje, jak byla funkce vytvořena; interpret zná sémantiku obecných uzlů ZERO, SUCCESSOR, PROJECTION, COMPOSITION, COMBINATION, RECURSION a MINIMIZATION, ale nezná speciální význam jmen ADD, MULTIPLY, QUOTIENT apod. Pojmenovaná funkce je jen odkaz na již sestavený AST.

Nejdůležitější zákaz ze zadání

ADD, MULTIPLY, POWER a podobné funkce nesmějí být speciální AST uzly s ručně napsaným algoritmem. Smějí být pojmenovanými odkazy na AST sestavený z povolených konstrukcí.

P.2 Doporučený objektový model AST

Následující návrh je jazykově nezávislý. Konkrétní třídy/recordy se mohou lišit, ale každá funkce by měla umět určit svou aritu a měla by být validovatelná ještě před evaluací.

```
FunctionNode
  arity() -> int
  validate(environment) -> ValidationResult

Zero(arity)
Successor
Projection(arity, index)
Composition(outer, inners[])
Combination(parts[])
Recursion(base, step)
Minimization(body)
Reference(name)
```

Doporučené invarianty: ZERO má explicitní aritu n a vrací 0 pro libovolnou n-tici; SUCCESSOR má aritu 1; PROJECTION(n,i) vyžaduje $1 \le i \le n$; COMPOSITION(g,h_1,...,h_k) vyžaduje k = arita(g) a všechny h_i mají stejnou vstupní aritu; COMBINATION(f_1,...,f_m) vyžaduje stejnou vstupní aritu všech složek a vrací jejich výsledky jako m-tici; RECURSION(base,step) kontroluje kompatibilitu arit podle schématu primitivní rekurze; MINIMIZATION(body) očekává alespoň jeden argument navíc pro hledanou proměnnou.

Arita primitivní rekurze

Je-li výsledná funkce f : N^(k+1) → N definována f(x,0)=g(x) a f(x,y+1)=h(x,y,f(x,y)), potom g má aritu k a h aritu k+2. AST RECURSION(g,h) tedy reprezentuje funkci arity k+1. Tato kontrola je důležitá pro parser i validaci.

Arita minimalizace

Je-li body g : N^(k+1) → N a minimalizujeme poslední argument, pak MINIMIZATION(g) reprezentuje funkci N^k → N. Evaluátor zkouší y=0,1,2,... a volá g(x,y), dokud nedostane 0 nebo nevyčerpá výpočetní limit.

P.3 Sémantika evaluace jednotlivých uzlů

Uzel	Sémantika	Kontrola

ZERO(n)	na n argumentech vrátí 0	počet argumentů = n
SUCCESSOR	S(x)=x+1	právě 1 argument
PROJECTION(n,i)	vrátí i-tou složku vstupu	$1 \le i \le n$ a n argumentů
COMPOSITION(g,h_1,…,h_k)	nejprve vyhodnotí všechna h_i na témže vstupu, jejich výsledky předá g	arity h_i stejné; počet h_i = arita g
COMBINATION(f_1,…,f_m)	vyhodnotí všechny f_i na témže vstupu a vrátí m-tici výsledků	všechny f_i mají stejnou vstupní aritu
RECURSION(g,h)	iteruje od základního případu po rekurzivní parametr	arita h = arita g + 2
MINIMIZATION(g)	zkouší y=0,1,… do první nuly	musí respektovat stepLimit
REFERENCE(name)	vyhledá AST v symbolické tabulce	jméno musí existovat; vhodné hlídat cyklické reference

Kombinace vs. COMBINE v zadání

Oficiální stránka používá pojem kombinace jako požadovanou schopnost, samostatný operátor COMBINE označuje na jednom místě jako nepovinný, ale v seznamu povinných testů se objevuje COMBINATION. Nejbezpečnější implementace pro XTILO je proto podporovat explicitní Combination uzel (nebo parserovou zkratku, která se před evaluací převede na ekvivalentní strukturu) a mít pro něj automatizovaný test. Pokud uzel skutečně vrací m-tici, je vhodné mít interní typ EvalValue = Scalar | Tuple; u běžných PRF uzlů je výsledkem Scalar. Jeho sémantiku je nutné přesně popsat v README.

Pseudokód obecné evaluace

```
evaluate(node, args, ctx):
    ctx.consumeStep()
    ctx.enter()
    try:
        validateRuntimeArity(node, args)
        match node:
            Zero(n):        return 0
            Successor:      return args[0] + 1
            Projection(n,i):return args[i-1]
            Composition(g, hs):
                values = [evaluate(h, args, ctx) for h in hs]
                return evaluate(g, values, ctx)
            Combination(fs):
                return [evaluate(f, args, ctx) for f in fs]
            Recursion(base, step):
                x = args[:-1]; y = args[-1]
                value = evaluate(base, x, ctx)
                for j in 0 .. y-1:
                    value = evaluate(step, x + [j, value], ctx)
                return value
            Minimization(body):
                for y = 0,1,2,...:
                    if evaluate(body, args + [y], ctx) == 0: return y
            Reference(name):
```

```
                return evaluate(environment[name], args, ctx)
    finally:
        ctx.leave()
```

Implementační poznámka

Pseudokód není součástí původních skript; je to návrhové doporučení pro projekt. Smyslem je, aby evaluátor zůstal obecný a aby žádná větev neobsahovala speciální logiku pro ADD, MULTIPLY atd.

P.4 EvaluationContext a výpočetní limit

Oficiální zadání vyžaduje vlastní výpočetní limit; timeout operačního systému nestačí. Kontext má sledovat alespoň počet kroků, maximální hloubku a limit kroků. Je vhodné definovat krok deterministicky a stejné pravidlo uvést v README.

Událost	Doporučené počítání
vstup do libovolného AST uzlu	+1 krok
vyhodnocení poduzlu	započítá se samostatně jeho vlastními kroky
jedna iterace rekurze	náklady evaluace step uzlu; případně +1 režie, pokud to dokumentace výslovně říká
jeden kandidát y v minimalizaci	náklady evaluace body; případně +1 režie podle zvolené konvence

Výsledek musí odlišovat normální hodnotu, překročení limitu, nevalidní AST a neplatné argumenty. Důležité: STEP_LIMIT_EXCEEDED není důkaz, že matematická funkce je nedefinovaná. Je pouze důkaz, že v daném rozpočtu nebyl výpočet dokončen.

```
EvalValue = SCALAR(BigInteger) | TUPLE(List<BigInteger>)

Result = VALUE(EvalValue)
       | STEP_LIMIT_EXCEEDED
       | INVALID_FUNCTION(message)
       | INVALID_ARGUMENTS(message)
```

P.5 Od teorie k AST: základní odvozené funkce

Níže jsou konstrukce, které jsou buď přímo ve VYSL2, nebo přímo v zadání semestrální práce. Zápis REFERENCE(NAME) znamená odkaz na pojmenovaný AST, nikoli speciální implementaci.

Konstanty

Konstantu 0 poskytuje ZERO(n). Konstantu m>0 lze vytvořit m-násobnou aplikací SUCCESSOR na nulovou funkci odpovídající arity. Prakticky je možné mít pomocný konstruktor const(n,m), který pouze VYGENERUJE tento AST; nesmí při evaluaci vracet m zkratkou.

```
ONE_2 = COMPOSITION(SUCCESSOR, ZERO(2))
TWO_2 = COMPOSITION(SUCCESSOR, ONE_2)
```

ADD - sčítání

$$add(x,0)=x \quad add(x,y+1)=S(add(x,y))$$

```
DEFINE ADD =
RECURSION(
    PROJECTION(1,1),
    COMPOSITION(
        SUCCESSOR,
        PROJECTION(3,3)
    )
)
```

PRED - předchůdce

$$pred(0)=0 \quad pred(y+1)=y$$

```
DEFINE PRED =
RECURSION(
    ZERO(0),
    PROJECTION(2,1)
)
```

Pokud zvolený datový model nepovoluje ZERO(0), lze ekvivalentně zavést nulovou 0-ární funkci podle VYSL2. Důležité je konzistentně dokumentovat, jak je 0-ární funkce reprezentována v AST a parseru.

SUB / monus - ořezané odečítání

$$sub(x,0)=x \quad sub(x,y+1)=pred(sub(x,y))$$

```
DEFINE SUB =
RECURSION(
    PROJECTION(1,1),
    COMPOSITION(
        REFERENCE(PRED),
        PROJECTION(3,3)
    )
)
```

MULTIPLY - násobení

$$mul(x,0)=0 \quad mul(x,y+1)=add(mul(x,y),x)$$

```
DEFINE MULTIPLY =
RECURSION(
    ZERO(1),
    COMPOSITION(
        REFERENCE(ADD),
        PROJECTION(3,1),
        PROJECTION(3,3)
    )
)
```

EQUIVALENCE / EQ - rovnost

VYSL2 používá charakteristickou funkci rovnosti s hodnotou 1 právě tehdy, když x=y, a 0 jinak. Jedna zdrojová konstrukce je:

$$eq(x,y) = 1 \ominus ((y \ominus x) + (x \ominus y))$$

Projektově ji lze sestavit pomocí již pojmenovaných SUB, ADD a konstanty 1. Výsledkem je stále AST složený ze základních konstrukcí; Reference pouze sdílejí dříve vytvořené podstromy.

```
DEFINE EQ =
COMPOSITION(
    REFERENCE(SUB),
    REFERENCE(ONE_2),
    COMPOSITION(
        REFERENCE(ADD),
        COMPOSITION(REFERENCE(SUB), PROJECTION(2,2), PROJECTION(2,1)),
        COMPOSITION(REFERENCE(SUB), PROJECTION(2,1), PROJECTION(2,2))
    )
)
```

QUOTIENT - celočíselný podíl

VYSL2 definuje quo(x,y)=floor(x/y) pro y≠0 a quo(x,0)=0 a dává primitivně rekurzivní schéma rekurzivní podle prvního argumentu x. Protože náš RECURSION rekurzuje podle POSLEDNÍHO argumentu, je praktické zavést pomocnou funkci q(y,x)=quo(x,y) a na konci argumenty prohodit projekcemi.

$$q(y,0)=0$$

$$q(y,x+1)=q(y,x)+eq(x+1,\ mul(q(y,x),y)+y)$$

```
DEFINE Q_INTERNAL = RECURSION(
    ZERO(1),
    COMPOSITION(
        REFERENCE(ADD),
        PROJECTION(3,3),
        COMPOSITION(
            REFERENCE(EQ),
            COMPOSITION(SUCCESSOR, PROJECTION(3,2)),
            COMPOSITION(
                REFERENCE(ADD),
                COMPOSITION(REFERENCE(MULTIPLY), PROJECTION(3,3), PROJECTION(3,1)),
                PROJECTION(3,1)
            )
        )
    )
)

DEFINE QUOTIENT = COMPOSITION(
    REFERENCE(Q_INTERNAL),
    PROJECTION(2,2),
    PROJECTION(2,1)
)
```

Kontrola na příkladu

Pro quotient(7,3) má Q_INTERNAL dostat (3,7). Rekurence zvýší kvocient při x+1 = (q+1)·y, tedy u 3 a 6, výsledkem je 2. Pro y=0 zdrojová definice vrací 0.

MODULO

Původní VYSL2 v mapované části nedává samostatnou definici modulo. Pro projekt lze z již zkonstruovaných funkcí odvodit standardní zbytek pomocí ořezaného odečtení:

$$mod(x,y) = sub(x,\ mul(quotient(x,y),\ y))$$

Toto je návrhové doplnění. S konvencí quotient(x,0)=0 vychází mod(x,0)=x; tuto hraniční hodnotu je vhodné výslovně uvést v dokumentaci a testech.

Ternární Σ a Π

Oficiální stránka mezi povinnými funkcemi uvádí symboly Σ(x,y,z) a Π(x,y,z), ale v textu stránky jejich sémantiku dále nerozepisuje. Proto je nelze ze zadání bezpečně ztotožnit bez další informace. Pokud vyučující zamýšlí ternární součet a součin, přirozená konstrukce je následující; v README ji označ jako zvolenou interpretaci nebo ji ověř s vyučujícím.

$$\Sigma_3(x,y,z) = add(add(x,y),z)$$

$$\Pi_3(x,y,z) = mul(mul(x,y),z)$$

```
SUM3 = COMPOSITION(ADD,
    COMPOSITION(ADD, P3_1, P3_2),
    P3_3)

PRODUCT3 = COMPOSITION(MULTIPLY,
    COMPOSITION(MULTIPLY, P3_1, P3_2),
    P3_3)
```

Zdrojová hranice

Názvy povinných funkcí pocházejí z oficiálního zadání. Konkrétní odvození ADD/MULTIPLY je výslovně v zadání; PRED, SUB, EQ a QUOTIENT mají oporu ve VYSL2. MODULO a výklad symbolů Σ/Π jsou zde uvedeny jako návrhové doplnění, protože dodané zdroje je v této podobě nedefinují.

P.6 Parser: text -> AST

Parser by měl pouze vytvořit syntaktický strom a následně spustit strukturální validaci. Nemá vyhodnocovat matematický zápis typu x+y ani z názvu ADD odvozovat speciální význam.

```
program      := statement*
statement    := define | evaluate | stepLimit
define       := "DEFINE" IDENT "=" expr
evaluate     := "EVALUATE" IDENT "(" numbers? ")"
expr         := zero | successor | projection | composition
              | combination | recursion | minimization | reference
zero         := "ZERO(" INT ")"
successor    := "SUCCESSOR"
projection   := "PROJECTION(" INT "," INT ")"
composition  := "COMPOSITION(" expr ("," expr)+ ")"
combination  := "COMBINATION(" expr ("," expr)+ ")"
recursion    := "RECURSION(" expr "," expr ")"
minimization := "MINIMIZATION(" expr ")"
reference    := "REFERENCE(" IDENT ")"
```

Statické validační chyby, které mají být zachyceny před během

- PROJECTION(3,4): index 4 přesahuje aritu 3.
- COMPOSITION: počet vnitřních funkcí neodpovídá aritě vnější funkce.
- COMPOSITION: vnitřní funkce nemají stejnou vstupní aritu.
- COMBINATION: složky nemají stejnou vstupní aritu.
- RECURSION: step nemá aritu base+2.
- MINIMIZATION: body nemá alespoň jednu vstupní složku navíc.
- REFERENCE: neexistující jméno; vhodné je také odmítnout cyklické definice $A \to B \to A$, pokud interpret nepodporuje rekurzivní jmenné vazby.

P.7 Pojmenované funkce a symbolická tabulka

Doporučená struktura je mapování jména na již validovaný AST. Jméno samo o sobě nemá výpočetní sémantiku: REFERENCE(ADD) pouze vyhledá strom uložený pod ADD. Tím lze složité stromy čitelně skládat a zároveň splnit zákaz speciálních implementací.

```
environment = {
    "ADD"      -> <Recursion AST>,
    "PRED"     -> <Recursion AST>,
    "SUB"      -> <Recursion AST>,
    "MULTIPLY" -> <Recursion AST>,
    "EQ"       -> <Composition AST>,
    ...
}
```

P.8 Testovací strategie

Oficiální zadání vyžaduje automatizované testy základních konstrukcí i povinných odvozených funkcí. Kromě běžných případů je důležité testovat arity, nulu a chování step limitu.

Oblast	**Minimální testy**
ZERO / SUCCESSOR	ZERO(0), ZERO(n) na správné aritě, successor $0 \to 1$ a velké BigInteger
PROJECTION	první/poslední složka; index 0; index > n; nesprávný počet argumentů
COMPOSITION	jednoduché $S \circ S$; více vnitřních funkcí; nekompatibilní arity
COMBINATION	více funkcí stejné arity; vrácené m-tice; nekompatibilní

	arity
RECURSION	ADD na nule i kladných hodnotách; MULTIPLY s nulou; hlubší iterace
MINIMIZATION	případ s řešením 0; řešení >0; případ překračující stepLimit
Derived	povinné Σ, quotient, equivalence, modulo, Π; dále SUB, EQ a helpery včetně hraničních vstupů
Parser	validní strom; chybějící závorka; neplatná projekce; neznámá reference
Context	přesný počet kroků pro malé příklady; maximumDepth; STEP_LIMIT_EXCEEDED

P.9 Individuální funkce podle studentského čísla

Oficiální zadání používá poslední tři číslice studentského čísla jako parametry a,b,c. Z nich vzniknou tři pomocné funkce R_b(x,y), M_c(x,y) a H_b(x,y) a výsledná kompozice T_a. Důležité je, že R musí být skutečně postavena primitivní rekurzí a M musí skutečně obsahovat MINIMIZATION - nestačí použít ekvivalentní přímý algoritmus.

1. Zapiš poslední tři číslice jako a,b,c a z oficiální tabulky zadání vyber konkrétní tvar T_a, rekurentní definici R_b, tělo G_c pro minimalizaci a pomocnou H_b.
2. Nejdřív postav a otestuj společnou knihovnu ZERO/S/PROJECTION/ADD/PRED/SUB/MULTIPLY/EQ/QUOTIENT/MODULO.
3. R_b sestav přímo jako RECURSION(base,step) podle předepsané rekurentní definice, i kdyby stejná matematická funkce už existovala jako pojmenovaný helper.
4. G_c sestav kompozicí existujících AST; potom M_c = MINIMIZATION(G_c).
5. H_b sestav běžnou kompozicí helperů.
6. Výslednou F sestav kompozicí T_a nad R_b, M_c a H_b.
7. Zvol malé testovací vstupy, pro které ručně vypočítáš R, M, H a F, a porovnej s interpretem.
8. V README uveď matematický zápis, výsledný AST, argument terminace/definovanosti M a experiment s alespoň dvěma stepLimit hodnotami.

Bez studentského čísla

Do skript nevkládám konkrétní individuální funkci, protože tvé poslední tři číslice zde nemám. Postup je ale úplný: až budou známy a,b,c, stačí z oficiální tabulky dosadit příslušné čtyři tvary a sestavit je pomocí uvedených AST konstrukcí.

P.10 Struktura projektu a README

Zadání požaduje jeden ZIP s README, zdroji, testy, definicemi povinných i individuální funkce a příklady vstupu/výstupu. Projekt musí být přeložitelný bez IDE a testy spustitelné standardním build příkazem. Praktická doporučená struktura:

```
PRF_<student_id>/
  README.md
  src/
  tests/
  functions/
    required.txt
    individual.txt
  examples/
    input.txt
```

```
  output.txt
<build file>
```

README checklist

- identifikace studenta a projektu;
- přesný příkaz pro build/spuštění a pro testy;
- gramatika vstupu a ukázka;
- objektový model AST a pravidla arit;
- přesná definice jednoho výpočetního kroku a step limitu;
- odvození individuální funkce z a,b,c;
- analýza definovanosti/terminace minimalizace;
- experimentální výsledky (hodnota, počet kroků, případně maximumDepth);
- známá omezení a zvolené konvence, zejména chování quotient/modulo pro dělitele 0 a význam Σ/Π, pokud není potvrzen jinak;
- potvrzení, že functions/required.txt obsahuje všechny povinné odvozené funkce a functions/individual.txt úplnou individuální definici;
- před odevzdáním zkontrolovat aktuální požadovaný název ZIPu a předmět e-mailu podle oficiální stránky zadání.

P.11 Nejčastější chyby, kterým se vyhnout

Chyba	Proč je problém
AddFunction vrací x+y	porušuje zákaz speciální implementace odvozené funkce
Parser přijímá x+y	zadání chce strukturální AST, ne interpret matematických výrazů
Step limit = timeout procesu	zadání vyžaduje vlastní deterministické počítání kroků
STEP_LIMIT_EXCEEDED = nedefinovaná funkce	logicky neplatný závěr; jen nevíme, zda by výpočet později skončil
Reference ADD bez uloženého AST	jméno nesmí nahrazovat syntaktickou definici
Rekurze s chybnými aritami	step musí mít dvě složky navíc: index rekurze a předchozí výsledek
Minimalizace přeskočí nedefinované dřívější g(x,z)	odporuje definici μ ve VYSL2; všechny předchozí hodnoty musí být definované
Individuální R nahrazena hotovým helperem	zadání vyžaduje, aby předepsaná R byla sama zkonstruována povinnou primitivní rekurzí

Co musíš umět k obhajobě

Nakreslit AST alespoň pro ADD a jednu složitější funkci; vysvětlit kontrolu arit; krok po kroku projít evaluaci RECURSION a MINIMIZATION; vysvětlit, proč step limit není důkaz nedefinovanosti; ukázat, že pojmenované funkce jsou jen reference na AST; odvodit vlastní R/M/H/F z a,b,c a vysvětlit, kde přesně je použita rekurze a minimalizace.

Zdrojová mapa

Teoretický základ: VYSL2 kap. 1-4, zejména počáteční funkce, primitivní rekurze, funkce pred/monus/eq/quo a minimalizace. Projektové požadavky: oficiální stránka kurzu 7TILO / XTILO „Semestrální práce - Interpret PRF“, ověřeno 28. 9. 2026. Návrh objektového modelu, pseudokód evaluace, parserová gramatika, modulo a testovací strategie jsou autorské doplnění pro realizaci projektu.

7TILO_9 - Důkazy, tabla, dedukce a rezoluce ve výrokové logice

9.1 Sémantika vs. formální důkaz

Sémantický přístup zkoumá pravdivost formulí v ohodnoceních/modelech. Formální důkaz pracuje syntakticky: má přesně určený jazyk, výchozí formule a odvozovací pravidla. Dvě klíčové vlastnosti důkazové metody jsou korektnost a úplnost.

- Sémantická korektnost: co je formálně dokazatelné, je sémanticky platné.
- Sémantická úplnost: co je sémanticky platné, lze danou metodou formálně dokázat.

9.2 Přímé a nepřímé důkazy

Přímý důkaz postupuje od předpokladů k závěru. Nepřímý důkaz přidá negaci požadovaného závěru a odvozuje spor. Pro logickou platnost formule A tak lze dokazovat nesplnitelnost ¬A. Pro logický důsledek $\Gamma \models A$ lze testovat nesplnitelnost $\Gamma \cup \{\neg A\}$.

$$\Gamma \models A \text{ právě když } \Gamma \cup \{\neg A\} \text{ je nesplnitelná}$$

9.3 Sémantické tablo ve výrokové logice

Tablo systematicky rozkládá složené formule na jednodušší. Formule typu α přidávají obě komponenty do stejné větve; formule typu β větví strom na dvě možnosti. Větev se uzavírá, obsahuje-li komplementární pár literálů p a ¬p.

Typ	Intuice	Příklad rozkladu
α	obě části musí platit současně	$A \wedge B \Rightarrow A, B$; $\neg(A \vee B) \Rightarrow \neg A, \neg B$
β	stačí jedna z alternativ	$A \vee B \Rightarrow$ větev A | větev B; $\neg(A \wedge B)$ obdobně

Formule je logicky platná, uzavře-li se tablo její negace. Otevřená větev poskytuje návod na model, ve kterém je testovaná formule splnitelná.

9.4 Klauzule a rezoluční pravidlo

Rezoluce pracuje s klauzulemi - disjunkcemi literálů. Máme-li dvě klauzule obsahující komplementární literály l a ¬l, můžeme je „zrušit“ a spojit zbytek:

$$(l \vee C_1), (\neg l \vee C_2) \implies C_1 \vee C_2$$

Výsledná klauzule je rezolventa. Rezoluční důkaz je posloupnost klauzulí, kde každá klauzule je buď výchozí, nebo rezolventa dřívějších klauzulí.

9.5 Rezoluční popření

Při nepřímém důkazu závěru A přidáme ¬A k premisám, vše převedeme do klauzulární formy a aplikujeme rezoluci. Pokud odvodíme prázdnou klauzuli □, výchozí množina je nesplnitelná, takže původní závěr skutečně plyne.

$$\text{premisy} + \neg\text{závěr} \implies^* \square$$

9.6 Doplnění pro pochopení - krátký příklad rezoluce

Z premis $(p \vee q)$, $(\neg p \vee r)$, ¬q a ¬r lze odvodit spor: z $(p \vee q)$ a ¬q dostaneme p; z $(\neg p \vee r)$ a p dostaneme r; s ¬r vznikne prázdná klauzule.

9.7 Úplná α a β pravidla podle tabulek LZUI2

Formule	Typ	Rozklad podle LZUI2
¬¬A	α	A

$A_1 \wedge A_2$	α	A_1, A_2 do stejné větve
$\neg(A_1 \vee A_2)$	α	$\neg A_1$, $\neg A_2$ do stejné větve
$\neg(A_1 \rightarrow A_2)$	α	A_1, $\neg A_2$ do stejné větve
$\neg(A_1 \leftarrow A_2)$	α	$\neg A_1$, A_2 do stejné větve
$A_1 \leftrightarrow A_2$	α	$(A_1 \rightarrow A_2)$, $(A_1 \leftarrow A_2)$ do stejné větve
$B_1 \vee B_2$	β	větev B_1 | větev B_2
$\neg(B_1 \wedge B_2)$	β	větev $\neg B_1$ | větev $\neg B_2$
$B_1 \rightarrow B_2$	β	větev $\neg B_1$ | větev B_2
$B_1 \leftarrow B_2$	β	větev B_1 | větev $\neg B_2$
$\neg(B_1 \leftrightarrow B_2)$	β	větev $\neg(B_1 \rightarrow B_2)$ | větev $\neg(B_1 \leftarrow B_2)$

9.8 Klauzulární forma před rezolucí

Rezoluce očekává konjunkci klauzulí. Praktický převod výrokové formule: odstraň ↔ a →, zatlač negace dovnitř De Morganovými pravidly, distributivitou převeď na CNF a jednotlivé konjunkty chápej jako množinu klauzulí. Tautologické klauzule obsahující p i ¬p lze při hledání sporu zahodit.

9.9 Prázdná klauzule

Prázdná klauzule □ nemá žádný literál a je nesplnitelná. V rezolučním popření je proto přesně hledaným syntaktickým svědkem sporu. Odvození □ z premis + negace závěru dokazuje, že tato množina nemá model.

Co musíš umět

Rozlišit sémantickou korektnost a úplnost, přímý/nepřímý důkaz, aplikovat úplnou sadu běžných α/β pravidel, poznat uzavřenou větev, převést jednoduchou formuli do CNF/klauzulí a provést rezoluční krok až k prázdné klauzuli.

Zdrojová mapa

LZUI1: kap. 5 Tablové a rezoluční nepřímé důkazy a kap. 6 Dedukce ve VL. LZUI2: kap. 1 Rezoluční důkazy ve VL a kap. 3 Tablové formální důkazy ve VL (s. 5-15 a 26-33).

7TILO_10_1 / 10_2 - Rezoluce a tabla v predikátové logice

10.1 Co přidává predikátová logika

Predikátová logika přidává termy, predikáty a kvantifikátory. Důkazové metody proto musí řešit nejen logické spojky, ale i náhrady proměnných termy a vazbu kvantifikátorů.

Doplnění pro pochopení

Term je výraz označující objekt: proměnná x, konstanta a nebo $f(t_1,...,t_n)$. Atom $p(t_1,...,t_n)$ tvrdí vztah mezi termy. ∀x A znamená „pro každý x“, ∃x A „existuje x“. Proměnná může být volná nebo vázaná kvantifikátorem.

10.2 Predikátová rezoluce: potřebné kroky

Před vlastní rezolucí je třeba formuli převést do vhodné klauzulární podoby. LZUI2 postupně probírá unifikaci, prenexní normální formu, Skolemizaci a predikátovou rezoluci.

Unifikace

Unifikace hledá substituci, která učiní dva atomy shodnými. Například p(x,a) a p(b,y) lze sjednotit substitucí {x/b, y/a}. Ne každá dvojice atomů je unifikovatelná; nesmí vzniknout konflikt funkčních symbolů ani nekonečný term.

Prenexní forma a Skolemizace

Prenexní normální forma přesune kvantifikátory před matici formule. Existenční kvantifikátory se při Skolemizaci eliminují zavedením nových Skolemových konstant nebo funkcí závislých na dříve univerzálně kvantifikovaných proměnných. Skolemizace obecně zachovává splnitelnost, ne nutně logickou ekvivalenci v původním jazyce.

Predikátové rezoluční pravidlo

Je-li možné dvojici komplementárních literálů sjednotit substitucí θ, nejprve aplikujeme θ a pak provedeme rezoluci. Schéma je obdobné výrokové rezoluci, ale doplněné unifikátorem.

$$(L \vee C_1),\ (\neg L' \vee C_2),\ \theta = \mathrm{mgu}(L,L') \implies (C_1 \vee C_2)\theta$$

10.3 Predikátové sémantické tablo

Pravidla α a β zůstávají v principu stejná jako ve výrokové logice. Kvantifikátory vyžadují dva další typy pravidel γ a δ.

Typ	Role	Intuice
α	nevětví	rozložení spojek, jejichž části musí být ve stejné větvi
β	větví	alternativy do dvou větví
γ	instance s již dostupnou konstantou	formuli lze instanciovat opakovaně podle potřeby
δ	instance s novou konstantou	zavede nový svědek/Skolemovu konstantu

Větev je uzavřená, obsahuje-li komplementární pár atomických formulí. U predikátové logiky mohou být tabla nekonečná, protože γ-pravidlo může vyžadovat opakované generování instancí.

10.4 Korektnost a úplnost

Zdroj uvádí korektnost: uzavřené tablo znamená nesplnitelnost. Úplnost: je-li predikátová formule logicky platná, existuje uzavřené tablo její negace. Praktická konstrukce může být obtížnější než u výrokové logiky, protože obecná predikátová logika není rozhodnutelná.

10.5 Přesná γ a δ pravidla tabla podle LZUI2

Typ formule	Pravidlo podle LZUI2	Konstanta

∀x A(x)	γ: ponech ∀xA(x) a přidej A(c)	c je konstanta již se vyskytující v jazyce / dostupná pro instanci
¬∃x A(x)	γ: ponech ¬∃xA(x) a přidej ¬A(c)	c je konstanta již se vyskytující v jazyce / dostupná pro instanci
∃x A(x)	δ: ponech ∃xA(x) a přidej A(d)	d je nová konstanta, dosud se v jazyce nevyskytující
¬∀x A(x)	δ: ponech ¬∀xA(x) a přidej ¬A(d)	d je nová konstanta, dosud se v jazyce nevyskytující

V konvenci LZUI2 se po aplikaci γ i δ pravidla původní kvantifikovaná formule v následujícím uzlu zachovává. Zdroj výslovně upozorňuje, že proto mohou být γ i δ pravidla na tutéž formuli aplikována opakovaně; u δ se přitom při každé aplikaci zavádí nová konstanta.

10.6 Doplnění pro pochopení - most mezi Skolemizací a δ-pravidlem

Oba postupy řeší existenci svědka. δ-pravidlo zavádí novou konstantu přímo při rozvoji tabla. Skolemizace před rezolucí nahrazuje existenční proměnnou novou konstantou nebo Skolemovou funkcí. Nejde o totožný algoritmus, ale stejná sémantická intuice „zaveď svědka“ pomáhá oba postupy pochopit.

10.7 Doplnění pro pochopení - most mezi unifikací a γ-instancí

Při rezoluci rozhoduje unifikace, jaké termy dosadit, aby se literály staly komplementární. V tablech se podobného cíle dosahuje vhodnou instancí univerzální γ-formule. V obou případech je cílem vytvořit syntaktickou dvojici, která umožní odvození/uzavření.

Co musíš umět

Provést jednoduchou unifikaci, vysvětlit smysl prenexní formy a Skolemizace, udělat predikátový rezoluční krok a bez záměny aplikovat přesná α/β/γ/δ pravidla tabla.

Zdrojová mapa

LZUI2: kap. 2 Rezoluční důkazy v predikátové logice (2.1-2.5, s. 16-25) a kap. 4 Tablové důkazy v predikátové logice (s. 34-41). LZUI1: kap. 8 jako doplňkový sémantický výklad.

7TILO_11 - Opakování a integrační přehled

Tato přednáška nepřidává nový formální aparát. Smyslem je spojit předchozí důkazové metody a procvičit volbu vhodného postupu. Nejefektivnější je opakovat nikoli kapitoly izolovaně, ale přes jeden společný úkol a sledovat, co by na něm dělaly různé metody.

11.1 Rozhodovací strom: jakou metodu použít

Úloha	Přirozený postup
Platnost jednoduché výrokové formule	tablo negace; případně pravdivostní tabulka
Logický důsledek ve VL z klauzulí	rezoluční popření nebo tablo premis + negace závěru
Predikátová dedukce	predikátové tablo nebo rezoluce po unifikaci/Skolemizaci
Potřebuji syntaktický formální důkaz v axiomatickém systému	Gentzen/Hilbert - viz 7TILO_12

11.2 Kontrolní úloha A - výroková rezoluce

Premisy: $p \rightarrow q$, $q \rightarrow r$, p. Dokaž r. Převod: $p \rightarrow q$ je $\neg p \vee q$; $q \rightarrow r$ je $\neg q \vee r$. Přidáme $\neg r$. Rezolucí s $\neg q \vee r$ dostaneme $\neg q$; s $\neg p \vee q$ dostaneme $\neg p$; s p dostaneme $\square$.

11.3 Kontrolní úloha B - tablo

Chceme ověřit platnost $(p \wedge (p \rightarrow q)) \rightarrow q$. Negujeme ji: $p \wedge (p \rightarrow q) \wedge \neg q$. α-rozklad vloží p, $p \rightarrow q$ a $\neg q$ do jedné větve. Implikace je β-typ: větev $\neg p$ se uzavře proti p, větev q proti $\neg q$. Všechny větve uzavřeny $\Rightarrow$ původní formule je platná.

11.4 Kontrolní úloha C - predikátový krok

Z $\forall x(P(x) \rightarrow Q(x))$ a $P(a)$ chceme $Q(a)$. Univerzální premisu instanciujeme na a, získáme $P(a) \rightarrow Q(a)$, a pak již postupujeme výrokově. Tato jednoduchá úloha ukazuje, že predikátová logika obsahuje výrokovou část plus práci s kvantifikátory.

Praktický plán opakování

Projdi cvičebnici v LZUI2, kap. 14: 14.1 rezoluce VL, 14.2 rezoluce PL, 14.3 tabla VL, 14.4 tabla PL. Důležitější než počet příkladů je umět u každého kroku pojmenovat použité pravidlo.

Zdrojová mapa

LZUI2: kap. 14 Cvičebnice, zejména 14.1-14.4 (s. 150-155); návazně LZUI1 kap. 5-8.

7TILO_12 - Axiomatické systémy: Gentzen G/G1 a Hilbert H/H1

12.1 Co je axiomatický systém

Axiomatický systém je určen třemi složkami: jazykem, axiómy a odvozovacími pravidly. Důkaz je konečná posloupnost formulí nebo seznamů formulí, kde každý krok je axióm nebo vznikl povoleným pravidlem z předchozích kroků.

Symbol $\vdash$ vyjadřuje syntaktickou dokazatelnost; symbol $\models$ sémantický logický důsledek/platnost. U korektního a úplného systému se oba pohledy v požadovaném rozsahu shodují.

12.2 Bezespornost, korektnost, úplnost

- Bezespornost: systém neumožňuje odvodit současně formuli a její negaci tak, aby se teorie stala triviální.
- Korektnost: $\vdash A \Rightarrow \models A$.
- Úplnost: $\models A \Rightarrow \vdash A$.

12.3 Gentzenovský systém G - výroková logika

G je založen na dualitě se sémantickými tably. Zatímco tablo rozkládá formuli při hledání modelu či sporu, Gentzenův systém používá duální pravidla k syntaktickému odvozování platných seznamů/formulí. Základ tvoří logické axiómy obsahující komplementární literály a duální α/β pravidla.

Výhodou je blízká vazba na stromové dokazování a transparentní vztah ke korektnosti a úplnosti. Speciálním případem G je klauzulární systém C, jehož klíčové β-pravidlo odpovídá rezoluci.

12.4 Hilbertovský systém H - výroková logika

H používá velmi malou sadu axiomových schémat a jedno základní pravidlo - modus ponens. LZUI2 uvádí tři klasická schémata:

$$A1: A \to (B \to A)$$

$$A2: (A \to (B \to C)) \to ((A \to B) \to (A \to C))$$

$$A3: (\neg B \to \neg A) \to (A \to B)$$

Z A a $A \to B$ se pravidlem modus ponens odvodí B. Pro praktické důkazy se používají také odvozená metapravidla, např. pravidlo dedukce a tranzitivity.

$$A,\ A \to B \implies B \quad \text{(modus ponens)}$$

12.5 G1 - predikátová varianta Gentzenova systému

G1 přenáší princip G do predikátové logiky. K α a β pravidlům přidává duální pravidla pro kvantifikátory, odpovídající γ a δ z predikátového tabla. Systém je v uvedeném zdroji prezentován jako sémanticky korektní a úplný.

12.6 H1 - predikátová varianta Hilbertova systému

H1 zachovává A1-A3 a přidává axiómy pro kvantifikátory: specifikaci univerzální formule a kvantifikaci implikace. Základní pravidla jsou modus ponens a generalizace.

$$A4: \forall x\ A \to A[t/x]$$

$$A5: \forall x(A \to B) \to (A \to \forall x B), \text{ pokud } x \text{ není volná v } A$$

$$\text{Generalizace: } \vdash A \Rightarrow \vdash \forall x\ A$$

12.7 G vs. H - co si zapamatovat

Systém	Charakter	Typická pravidla
G	stromově/sequentově orientovaný, dualita s tablem	duální α, β
H	málo axiomů, lineární posloupnost důkazu	modus ponens + pomocná pravidla

G1	predikátová verze G	α, β, γ, δ v duální formě
H1	predikátová verze H	A1-A5, modus ponens, generalizace

12.8 Důkaz v Hilbertově systému - jak číst jednotlivé řádky

Každý řádek musí mít zdůvodnění: je buď instancí axiomu, speciálním předpokladem, nebo vznikl z předchozích řádků pravidlem. U modus ponens musí být přítomny přesně tvary A a A→B. Důkaz v H proto není „sémantické vysvětlování“, ale mechanická syntaktická posloupnost.

12.9 Pravidlo dedukce

Velmi užitečné pomocné pravidlo převádí důkaz z dočasného předpokladu na implikaci. V jednoduchém výrokovém případě:

$$\Gamma, A \vdash B \quad \text{právě když} \quad \Gamma \vdash A \to B$$

Tento princip vysvětluje, proč lze v Hilbertově systému pracovat stylem „předpokládej A, odvoď B, uzavři důkaz implikací A→B“.

12.10 Predikátové rozšíření H1

A4 umožňuje specifikovat univerzální tvrzení na konkrétní term; A5 a pravidlo generalizace naopak umožňují řízeně zavádět univerzální kvantifikaci. Podmínky na volný výskyt proměnné nejsou technická drobnost - bez nich by pravidlo mohlo být sémanticky nekorektní.

Co musíš umět

Definovat axiomatický systém, odlišit $\vdash$ a $\models$, vysvětlit korektnost/úplnost/bezespornost, charakterizovat G vs. H a G1 vs. H1, znát A1-A5, modus ponens, generalizaci a princip pravidla dedukce.

Zdrojová mapa

LZUI2: kap. 5 Formální dedukce v axiomatických systémech; kap. 6 G; kap. 7 H; kap. 8 G1; kap. 9 H1 (s. 42-91).

7TILO_13 - Klauzulární logika a znalostní báze

13.1 Od predikátových formulí ke klauzulím

Klauzulární logika používá formule ve tvaru klauzulí a je vhodná pro automatizované odvozování a reprezentaci znalostí. LZUI2 rozlišuje syntaxi klauzulárního jazyka, jeho sémantiku, speciální typy klauzulí a znalostní báze.

13.2 Hornovy klauzule

Zvlášť důležité jsou Hornovy klauzule, které mají nejvýše jeden kladný literál v běžném disjunktivním zápisu. V pravidlovém zápisu je lze chápat jako „pokud platí antecedenty, pak platí konsekvent“. Jsou základem mnoha pravidlových a logických programovacích systémů.

13.3 Fakta, pravidla a dotazy

Znalostní báze může obsahovat fakta a pravidla. Fakt je tvrzení bez podmínky, například → člověk(sokrates). Pravidlo může mít podobu člověk(x) → smrtelný(x). Dotaz pak zkoumá, zda požadovaná klauzule plyne z báze.

13.4 Interpretace a model znalostní báze

Jazyk sám o sobě neurčuje význam symbolů. Interpretace přiřazuje konstantám objekty, predikátovým symbolům relace a funkčním symbolům funkce nad univerzem diskursu. Model znalostní báze je taková aplikovatelná struktura, ve které jsou všechny klauzule báze platné.

$$\text{S je model KB} \;\Leftrightarrow\; \text{každá klauzule KB je v S platná}$$

13.5 Logický důsledek znalostní báze

Klauzule Z je logickým důsledkem znalostní báze právě tehdy, když je platná ve všech modelech této báze. To je přesně sémantická definice dedukce aplikovaná na znalostní báze.

$$KB \models Z \;\Leftrightarrow\; \text{Z platí ve všech modelech KB}$$

13.6 Předpoklad uzavřeného světa

Zdroj zařazuje také předpoklad uzavřeného světa: to, co není ve znalostní bázi známé jako pravdivé (nebo z ní odvoditelné podle zvolené interpretace systému), se v určitých praktických systémech považuje za nepravdivé. Jde o dodatečný předpoklad, nikoli o obecný zákon klasické logiky.

> **Pozor**
>
> Klasická logika používá otevřený svět: z toho, že P neumíme dokázat, neplyne ¬P. Closed-world assumption tuto mezeru z praktických důvodů uzavírá a je proto ne-monotónním/implementačním předpokladem nad rámec čisté klasické sémantiky.

13.7 Odvozování v klauzulární logice

LZUI2 dále zavádí axiomatický systém C1 klauzulární logiky a rezoluci v tomto systému. Přímé a nepřímé odvozování pak propojuje znalostní bázi s důkazovými metodami z 7TILO_9 a 10.

13.8 Tři praktické tvary klauzulí

Tvar	Interpretace	Příklad
→ P(a)	fakt	→ člověk(sokrates)
P(x) → Q(x)	pravidlo	člověk(x) → smrtelný(x)
P(a) →	negativní tvrzení / zákaz	má_rád(královna,sněhurka) →

Obecná klauzule může mít více atomů na obou stranách, ale Hornovy tvary jsou pro automatizované odvozování zvlášť příjemné.

13.9 Dopředné a zpětné odvozování

Dopředné odvozování začíná fakty a opakovaně aplikuje pravidla, jejichž antecedenty jsou splněny. Zpětné odvozování začíná cílem a hledá pravidla, která jej mohou vytvořit, a z nich odvozuje podcíle. Tyto postupy souvisejí s H1 a znalostními bázemi, i když konkrétní implementace může používat rezoluci.

13.10 Konzistence znalostní báze

Je-li báze nekonsistentní, klasická logika dovoluje odvodit libovolný závěr. Proto zdroj opakovaně zdůrazňuje význam bezespornosti/konsistence. V praktickém systému je nutné odlišit „nemám informaci“ od skutečného rozporu P a ¬P.

Co musíš umět

Rozpoznat klauzuli/Hornovu klauzuli a praktické tvary fakt/pravidlo/negativní tvrzení, sestavit jednoduchou znalostní bázi, vysvětlit interpretaci, model, logický důsledek, dopředné/zpětné odvozování a closed-world assumption.

Zdrojová mapa

LZUI2: kap. 10 Klauzulární logika (s. 92-115), kap. 11 Znalostní báze a její modely (s. 116-128); návazně kap. 12-13 pro C1, rezoluci a odvozování.

7TILO_14 - Kvantové počítání: qubity, obvody, složitost, Grover a Shor

14.1 Prerekvizity: lineární algebra v minimálním rozsahu

Doplnění pro pochopení

Kvantový stav budeme zapisovat jako vektor. Potřebuješ vědět: komplexní číslo má reálnou a imaginární část; skalární součin ⟨φ|ψ⟩ měří „překryv“ vektorů; norma stavového vektoru musí být 1; matice reprezentuje lineární operátor; tensorový součin skládá více systémů. Hermitovsky sdružená matice U† vznikne transpozicí a komplexním sdružením.

14.2 Kvantová mechanika: kvantový stav, superpozice a měření

Zdroj reprezentuje kvantové stavy jako normované vektory v komplexním vektorovém prostoru. Diracova notace používá ket |ψ⟩. Stav může být superpozicí bázových stavů:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1$$

Koeficienty α a β jsou amplitudy. Při měření v dané bázi dostaneme výsledek 0 s pravděpodobností $|\alpha|^2$ a 1 s pravděpodobností $|\beta|^2$. Měření změní stav - po naměření 0 se stav zkolabuje do |0⟩.

14.3 Qubit a Blochova sféra

Qubit je kvantový analog bitu, ale před měřením může být v libovolné normalizované superpozici |0⟩ a |1⟩. Blochova sféra je geometrická reprezentace čistých stavů jednoho qubitu; globální fáze se na výsledcích měření neprojeví.

14.4 Více qubitů a kvantový registr

Více systémů se skládá tensorovým součinem. Pro n qubitů má stavový prostor rozměr 2^n a standardní bázi tvoří všechny n-bitové řetězce. Zdroj výslovně označuje systém n qubitů jako quantum register.

$$|x_1x_2\ldots x_n\rangle, \quad x_i \in \{0,1\}$$

Ne každý vícequbitový stav lze rozložit na tensorový součin samostatných qubitů. Takové stavy jsou provázané (entangled). Provázanost je charakteristickým kvantovým zdrojem, ale sama o sobě nezaručuje výpočetní zrychlení.

14.5 Unitární evoluce a reverzibilita

Kvantová evoluce bez měření je popsána unitárním operátorem U, pro který U†U=UU†=I, tedy $U^{-1}=U^\dagger$. Proto je taková transformace invertibilní. To je matematický základ reverzibility kvantových bran.

Obecnou klasickou funkci f nelze vždy implementovat přímo jako reverzibilní zobrazení x↦f(x), protože různé vstupy mohou mít stejný výstup. Standardní trik uchová vstup a výsledek přičte XORem do pomocného registru:

$$U_f\,|x,y\rangle = |x,\ y \oplus f(x)\rangle$$

14.6 Kvantové brány a obvody

Standardní model kvantového výpočtu je kvantový obvod. Začíná počátečním registrem, aplikuje posloupnost unitárních bran a nakonec měří. Celkový operátor je kompozicí bran $U = U_L \ldots U_2U_1$. Hloubka obvodu měří počet navazujících vrstev bran.

- Pauli-X: kvantový analog NOT, zamění |0⟩ a |1⟩.
- Hadamard H: vytváří/ruší rovnoměrnou superpozici; $H|0\rangle=(|0\rangle+|1\rangle)/\sqrt{2}$.
- CNOT: dvouqubitová řízená brána; spolu s jednoqubitovými branami je základem univerzálních obvodů.

14.7 Vyčíslitelnost a BQP

Kvantový počítač podle zdroje nerozhoduje problémy, které by byly v klasickém smyslu nevyčíslitelné. Změna přichází ve složitosti. BQP je třída rozhodovacích problémů řešitelných kvantovým výpočtem v polynomiálním čase s pravděpodobností správné odpovědi oddělenou od 1/2 konstantní rezervou.

$$P \subseteq BPP \subseteq BQP \subseteq PSPACE$$

Vztah BQP k NP není obecně znám tak, aby se dal jednoduše ztotožnit. Kvantový počítač proto není „stroj, který automaticky řeší všechny NP-úplné problémy rychle“.

14.8 Kvantový stroj: Quantum Turing Machine

Zdroj uvádí Quantum Turing Machine jako kvantovou analogii klasického TS: konečné množiny stavů a páskových symbolů jsou nahrazeny Hilbertovými prostory, přechody jsou unitární a pravděpodobnost se objeví při měření. Kvantový Turingův stroj a kvantový obvod mohou simulovat jeden druhý v odpovídajícím smyslu, takže představují ekvivalentní modely kvantového výpočtu.

14.9 Kvantová nadřazenost / quantum advantage

Doplnění - v hlavním zdroji není samostatná kapitola

Quantum supremacy označuje experimentální demonstraci kvantového výpočtu, který je pro zvolenou úlohu proveditelný na kvantovém zařízení a podle dostupných klasických metod mimo praktický dosah. Termín quantum advantage se dnes často používá obecněji a méně absolutně. Neznamená to, že kvantový počítač je rychlejší pro všechny úlohy. Formální teorie BQP a konkrétní speedupy (Grover, Shor) jsou teoretickou stránkou této otázky.

14.10 Groverův algoritmus

Grover řeší nestrukturované vyhledávání v prostoru N možností pomocí oracle. Začneme v rovnoměrné superpozici všech kandidátů, oracle změní fázi hledaného stavu a diffusion/amplitude-amplification krok zvýší jeho amplitudu. Po přibližně O(√N) iteracích měření vrátí hledaný prvek s vysokou pravděpodobností.

klasické vyhledávání: O(N) Grover: O(√N)

Zrychlení je kvadratické, nikoli exponenciální. Grover je proto důležitý jako obecný, ale omezený nástroj.

14.11 Quantum Fourier Transform (QFT)

QFT je kvantová verze diskrétní Fourierovy transformace implementovaná kvantovým obvodem. V Shorově algoritmu slouží k extrakci periodicity/fáze ze superpozice. Pro pochopení Shora není nutné odvozovat každý detail QFT, ale je nutné rozumět její roli jako změně báze, která zpřístupní periodickou strukturu měřením.

14.12 Shorův faktorizační algoritmus

Shor převádí faktorizaci složeného čísla N na problém hledání řádu: pro zvolené x hledá nejmenší kladné r takové, že x^r ≡ 1 (mod N). Kvantová část řeší period/order-finding s využitím Fourierovy transformace; klasický post-processing z nalezeného r získává kandidátní dělitele pomocí gcd.

Postup níže zachovává pět kroků uvedených v hlavním kvantovém zdroji:

1. Je-li N sudé, vrať faktor 2.
2. Je-li N = a^b pro nějaké a ≥ 1 a b ≥ 2, vrať netriviální základ a.
3. Zvol x z intervalu 1 až N-1. Pokud gcd(x,N)>1, vrať gcd(x,N) jako netriviální faktor.
4. Použij order-finding a najdi řád r čísla x modulo N. Je-li r sudé, x^(r/2) ≠ -1 (mod N) a některé z gcd(x^(r/2)-1,N), gcd(x^(r/2)+1,N) je netriviální, vrať tento faktor.
5. Jinak opakuj od kroku 3 s jinou volbou x.

Zdroj uvádí pro Shorův kvantový faktorizační algoritmus polynomiální čas v délce binárního zápisu N a zdůrazňuje, že kvantová část řeší order-finding, zatímco gcd a další kontroly jsou klasický post-processing. To je zásadní pro kryptosystémy založené na praktické obtížnosti faktorizace.

14.13 Aplikace

Zdroj má celou třetí část věnovanou aplikacím. Pro osnovu stačí rozlišovat několik směrů: kvantové procházky a Monte Carlo, optimalizace (včetně QAOA/annealingu), finanční aplikace, kvantová distribuce klíčů a bezpečnost, faktorizační algoritmy a kvantová simulace.

14.14 Základní matice bran

Pro jeden qubit se brány zapisují jako 2×2 unitární matice. Užitečné minimum:

X = [[0,1],[1,0]]

$$Z = [[1,0],[0,-1]]$$

$$H = (1/\sqrt{2})\ [[1,1],[1,-1]]$$

X prohodí bázové stavy, Z mění fázi $|1\rangle$ a H převádí mezi výpočetní a Hadamardovou bází. Dvakrát aplikovaný Hadamard dává identitu: $H^2=I$.

14.15 Měření a interference

Kvantové algoritmy nezískávají výhodu tím, že bychom „přečetli všechny stavy superpozice“. Jedno měření dá jediný výsledek. Algoritmus musí pomocí fází a interference potlačit amplitudy špatných výsledků a zesílit amplitudy užitečných výsledků. Groverova amplitude amplification je nejčistší příklad této myšlenky.

14.16 BQP není totéž co „exponenciální zrychlení“

BQP je třída problémů definovaná zdrojovými omezeními výpočtu, ne tvrzení o konkrétním poměru rychlostí. Některé kvantové algoritmy dávají exponenciální zrychlení pro určité formulace, Grover dává kvadratické a pro mnoho běžných úloh není znám relevantní kvantový speedup.

14.17 Grover geometricky

Stavový vektor lze rozdělit na složku ve směru hledaného stavu $|w\rangle$ a složku kolmo k němu. Oracle provede odraz fáze kolem jedné osy a diffusion operator druhý odraz. Dva odrazy dohromady znamenají rotaci směrem k $|w\rangle$. Po přibližně $\pi/4\cdot\sqrt{N}$ iteracích je amplituda cíle blízko maxima.

14.18 Doplnění pro pochopení - Shor: co je kvantové a co klasické

Část	Charakter
výběr a, gcd testy, závěrečné gcd	klasický výpočet
příprava superpozice a modulární exponentace	kvantový obvod
QFT / phase-period information	kvantová klíčová část
continued fractions / rekonstrukce r	klasický post-processing

Co musíš umět

Vysvětlit qubit, amplitudy a měření, n-qubitový registr a entanglement, popsat unitární/reverzibilní bránu a obvod, znát základní X/Z/H, definovat smysl BQP, charakterizovat kvantový TS, vysvětlit interferenci, speedup Grovera a rozdělení Shorova algoritmu na order-finding + klasický post-processing.

Zdrojová mapa

Quantum Computing via Randomized Algorithms (2025): kap. 1 Quantum Mechanics 101; 2.1 The qubit; kap. 5 Theoretical Computer Science 101; kap. 6 Quantum Computing 101 (zejména 6.1.4 BQP, 6.3 QTM, 6.4 circuits); 8.3 QFT; 9.1 Grover; 14.3.1 Shor factoring; aplikace v kap. 10-14. Quantum supremacy/advantage je doplněno vysvětlením mimo hlavní zdroj.

Dodatek A - Audit pokrytí osnovy

Níže je kontrolní matice, kterou lze použít i jako checklist před zkouškou. „Pokryto“ znamená, že bod osnovy má v těchto custom skriptech vlastní výklad a zdrojovou vazbu. „Doplněno“ znamená, že původní hlavní zdroj téma nepokrýval samostatně a byla přidána vysvětlující část.

Přednáška	Bod osnovy	Sekce skript	Zdroj	Stav
7TILO_1	množiny, relace, funkce	1.1	VYSL1 1.1/1.4/1.5	Pokryto
7TILO_1	problémy	1.2	VYSL1 kap. 2	Pokryto
7TILO_1	TS a konfigurace	1.3-1.6	VYSL1 3.1-3.2	Pokryto
7TILO_2	kódování/ ekvivalenty/NTS/ UTS	2.1-2.4	VYSL1 3.3-3.6	Pokryto
7TILO_2	nerozhodnutelnost	2.5	VYSL1 kap. 4	Pokryto
7TILO_3	složitost a asymptotika	3.1-3.4	VYSL1 8.1-8.3	Pokryto
7TILO_3	P/NP/redukce	3.5-3.8	VYSL1 8.4-8.6	Pokryto
7TILO_4	RAM	4	VYSL1 kap. 6	Pokryto
7TILO_5	počáteční funkce/kompozice/P R	5	VYSL2 kap. 1	Pokryto
7TILO_6	PRF/tabulkové/μ-rek./Ackermann	6	VYSL2 kap. 2	Pokryto
7TILO_7	minimalizace/ parciálně rekurzivní funkce ↔ TS/kódování konfigurace	7	VYSL2 kap. 3	Pokryto
7TILO_8	PL a vyjadřovací síla	8	VYSL1 7.2 + VYSL2 kap. 4	Pokryto
7TILO_9	přímé/nepřímé, tabla VL, rezoluce VL	9	LZUI1 5-6 + LZUI2 1,3	Pokryto
7TILO_10	rezoluce PL, tabla PL, αβγδ	10	LZUI2 2,4	Pokryto
7TILO_11	opakování	11	LZUI2 cvičebnice	Pokryto
7TILO_12	axiomatické systémy G/H/G1/H1	12	LZUI2 5-9	Pokryto
7TILO_13	klauzulární logika/znalostní báze	13	LZUI2 10-13	Pokryto
7TILO_14	QM, qubit, registr,	14.1-14.6	Quantum Computing	Pokryto

	obvod, reverzibilita		1,2,6	
7TILO_14	složitost/BQP/QTM	14.7-14.8	Quantum Computing 5-6	Pokryto
7TILO_14	quantum supremacy/advantage	14.9	mimo hlavní zdroj	Doplněno
7TILO_14	Grover/QFT/Shor/ aplikace	14.10-14.13	Quantum Computing 8,9,10-14	Pokryto

Dodatek B - Co bylo záměrně komprimováno nebo vynecháno

Tento seznam je důležitý pro transparentnost: neznamená, že jde o bezcenný obsah, pouze že není nutný pro přímé pokrytí zadané osnovy nebo je reprezentován kratším principem.

- Historické a motivační pasáže, které neopravují nebo nerozšiřují definici pojmu.
- Většina opakovaných cvičení a dlouhých ručních derivací; zachován je postup a reprezentativní příklad.
- VYSL1 kap. 5 Enumerace Turingových strojů a Riceova věta nejsou v zadané osnově uvedeny. Jsou však relevantním rozšířením 7TILO_2; pokud je vyučující vyžaduje, je vhodné je doplnit.
- VYSL1 8.7-8.8: podrobnější katalog NP-úplných a „nezvládnutelných“ problémů je zkrácen na princip NP-úplnosti a SAT.
- LZUI1 části věnované základní syntaxi/sémantice logiky jsou použity jen jako prerekvizita; hlavní důkazové metody přebírá LZUI2.
- LZUI2 programové podpory a velká část cvičebnice nejsou přepisovány celé.
- Kvantová kniha obsahuje řadu témat nad rámec 7TILO_14 (harmonické oscilátory, detailní kvantová informace, quantum walks, VQA, finance). V hlavním textu jsou z nich ponechány jen části potřebné pro osnovu a přehled aplikací.

Doporučení pro maximální jistotu

Pokud máš k dispozici oficiální otázky ke zkoušce nebo prezentace vyučujícího, poslední audit je vhodné provést ještě proti nim. Tím se ověří nejen pokrytí zveřejněné osnovy, ale i přesná hloubka, kterou vyučující očekává.

Dodatek C - Rychlý tahák pojmů

Pojem	Jednovětá definice
Rozhodnutelný problém	Existuje algoritmus, který se pro každý vstup zastaví se správnou odpovědí.
Konfigurace TS	Úplný okamžitý popis obsahu pásky, pozice hlavy a stavu.
Univerzální TS	Stroj simulující jiný stroj z jeho kódu a vstupu.
Redukce	Efektivní převod instancí jednoho problému na druhý se zachováním odpovědi.
P	Rozhodovací problémy řešitelné deterministicky v polynomiálním čase.
NP	Rozhodovací problémy řešitelné nedeterministicky v polynomiálním čase.
PRF	Funkce vytvořené z počátečních funkcí kombinací,

	kompozicí a primitivní rekurzí; jsou totální.
Minimalizace μ	Hledání nejmenšího y s g(x,y)=0; může vést k nedefinovanosti.
Rezoluce	Odvozovací pravidlo rušící komplementární literály ve dvou klauzulích.
Tablo	Stromový rozklad formule; uzavřená větev obsahuje spor.
Korektnost	Dokazatelnost implikuje sémantickou platnost.
Úplnost	Sémantická platnost implikuje dokazatelnost.
Model znalostní báze	Struktura, v níž jsou všechny klauzule báze platné.
Qubit	Normalizovaný stav $\alpha\|0\rangle+\beta\|1\rangle$ v C^2.
Unitární operátor	Invertibilní lineární operátor U s $U^\dagger U=I$.
BQP	Kvantově polynomiálně řešitelné rozhodovací problémy s omezenou chybou.
Grover	Kvantové nestrukturované vyhledávání v $O(\sqrt{N})$ oracle dotazech.
Shor	Kvantový polynomiální algoritmus pro faktorizaci přes period/order finding a QFT.

Dodatek D - Auditované opravy ve verzi v3

Tato verze navazuje na audit v2 a přidává praktickou XTILO kapitolu, která byla znovu kontrolována proti původním skriptům i aktuálnímu oficiálnímu zadání. Níže jsou změny, u nichž byla v předchozí verzi nalezena věcná nebo zdrojová nepřesnost.

Místo	**Oprava v auditované v3**
5.2 Projekce	Notace opravena na π^n_k; horní index = arita vstupu, dolní index = vybíraná složka. Příklad $\pi^3_2(7,6,4)=6$.
7.8-7.9 vztah funkcí a TS	Odstraněna nejednoznačná zkratka pro vztah primitivně/parciálně rekurzivních funkcí a TS; přesně uvedeny parciálně rekurzivní funkce ↔ Turingovy stroje. Funkce step je označena jako parciálně rekurzivní.
9.7 tabla ve VL	Doplněna úplná α/β tabulka z LZUI2 včetně ← a ↔.
10.5 tabla v PL	Podle konvence LZUI2 se po γ i δ ponechává původní kvantifikovaná formule; u δ se přidává nová konstanta.
2.8 HP → UHP	Konstrukce přeformulována přesně podle VYSL1: M′ simuluje M jen na vstupu rovném pevnému w, jinak se zastaví.
3.7 NP/NPTIME	Primární formulace změněna na znění používané ve VYSL1; certifikátový pohled je výslovně označen jako doplnění.

4.7 RAM odečtení	Zachycen tiskový překlep ve VYSL1: tabulka opakuje STORE, ale slovní význam je odečtení; v textu je to transparentně uvedeno.
Doplňující příklady	Vlastní příklady a moderní mosty jsou v nadpisech nebo blocích označeny jako Doplnění pro pochopení.
14.12 Shor	Druhý audit doplnil dva chybějící počáteční kroky zdrojového algoritmu (sudé N; perfektní mocnina) a opravil volbu x/gcd tak, aby odpovídala pěti krokům uvedeným ve zdroji.
XTILO praktikum	Přidána implementační kapitola k semestrální práci: AST, arity, obecná evaluace, step limit, parser, konstrukce odvozených funkcí, testy, workflow individuální funkce a odevzdání. Zdrojové vs. autorské části jsou výslovně rozlišeny.

Auditní zásada

Pokud se obecně známá terminologie nebo běžná učebnicová konvence liší od konkrétního zdroje, auditovaná verze v3 nejprve uvádí formulaci zdroje. Moderní nebo alternativní formulace je oddělena jako doplnění.

Bibliografická stopa zdrojů

S1 - VYSL1.pdf: Vyčíslitelnost a složitost, Hashim Habiballa.

S2 - VYSL2.pdf: Vyčíslitelnost a složitost 2, Viktor Pavliska.

S3 - LZUI1.pdf: Logika pro informatiky, Martin Kotyrba.

S4 - LZUI2.pdf: Logika pro umělou inteligenci, Martin Kotyrba.

S5 - Quantum Computing via Randomized Algorithms, Jakub Mareček, Georgios Korpas, Johannes Aspman, Springer, 2025.

Poznámka k číslování: rozsahy stran v textu vycházejí primárně z číslování kapitol/obsahu zdrojů; fyzické číslo stránky PDF se může lišit o úvodní listy.